Comp90038
Tutorial 6:
Week 7

Today’s Tutorial Flow

- Tutorial Questions
  - QuickSelect: Q42, Q43
  - Mergesort: Q44, Q48
  - Quicksort: Q45
  - Master Theorem: Q46

Tutorial Questions

Order of Discussion for today's workshop:

Q46, Q42,43,45, Q44 → Q48

Tutorial Questions

46. Use the Master Theorem to find the order of growth for the solutions to the following recurrences. In each case, assume $T(1) = 1$, and that the recurrence holds for all $n > 1$.

    (a) $T(n) = 4T(n/2) + n$

    (b) $T(n) = 4T(n/2) + n^2$

    (c) $T(n) = 4T(n/2) + n^3$

For the general case, assume we split the problem into ***b*** instances (each of size ***n/b***), of which ***a*** need to be solved:

$$T(n) = aT(n/b) + f(n)$$

where $f(n)$ expresses the time spent on dividing a problem into ***b*** sub-problems and combining the ***a*** results.

Tutorial Questions

46. Use the Master Theorem to find the order of growth for the solutions to the following recurrences. In each case, assume $T(1) = 1$, and that the recurrence holds for all $n > 1$.

    (a) $T(n) = 4T(n/2) + n$

    (b) $T(n) = 4T(n/2) + n^2$

    (c) $T(n) = 4T(n/2) + n^3$

For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d), d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with $T(1) = c$) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Tutorial Questions

46. Use the Master Theorem to find the order of growth for the solutions to the following recurrences. In each case, assume $T(1) = 1$, and that the recurrence holds for all $n > 1$.

(a) $T(n) = 4T(n/2) + n$

(b) $T(n) = 4T(n/2) + n^2$

(c) $T(n) = 4T(n/2) + n^3$

Solution to (a)

$$T(n) = 4T(n/2) + n$$

$a = 4, b = 2$, and $f(n) = n^1$, so $d = 1$.

$b^d = 2^1 = 2$.

Therefore, $a > b^d$.

So $T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_2 4}\right) = \Theta(n^2)$.

$$T(n) = \Theta(n^2)$$

For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d), d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with $T(1) = c$) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Tutorial Questions

46. Use the Master Theorem to find the order of growth for the solutions to the following recurrences. In each case, assume $T(1) = 1$, and that the recurrence holds for all $n > 1$.

(a) $T(n) = 4T(n/2) + n$

(b) $T(n) = 4T(n/2) + n^2$

(c) $T(n) = 4T(n/2) + n^3$

Solution to (b)

$$T(n) = 4T(n/2) + n^2$$

$a = 4, b = 2$, and $f(n) = n^2$, so $d = 2$.

$b^d = 2^2 = 4$.

Therefore, $a = b^d$.

So $T(n) = \Theta(n^d \log n) = \Theta(n^2 \log n)$.

$$T(n) = \Theta(n^2 \log n)$$

For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d), d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with $T(1) = c$) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Tutorial Questions

46. Use the Master Theorem to find the order of growth for the solutions to the following recurrences. In each case, assume $T(1) = 1$, and that the recurrence holds for all $n > 1$.

(a) $T(n) = 4T(n/2) + n$

(b) $T(n) = 4T(n/2) + n^2$

(c) $T(n) = 4T(n/2) + n^3$

Solution to (c)

$$T(n) = 4T(n/2) + n^3$$

$a = 4, b = 2$, and $f(n) = n^3$, so $d = 3$.

$b^d = 2^3 = 8$.

Therefore, $a < b^d$.

So $T(n) = \Theta(n^d) = \Theta(n^3)$.

$$T(n) = \Theta(n^3)$$

For integer constants $a \geq 1$ and $b > 1$, and function f with $f(n) \in \Theta(n^d), d \geq 0$, the recurrence

$$T(n) = aT(n/b) + f(n)$$

(with $T(1) = c$) has solutions, and

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Tutorial Ouestions

42. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```

```
function QuickSelect(A[·], lo, hi, k)
    s ← LomutoPartition(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QuickSelect(A, lo, s − 1, k)
        else
            QuickSelect(A, s + 1, hi, (k − 1) − (s − lo))
```

S+1th smallest element

Tutorial Ouestions

42. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

- We are after the median, and for an array of length 7, that means the fourth smallest element. Hence the algorithm's k is 4. QuickSelect will keep probing until it manages to place the pivot in position 4 − 1 = 3.

39	23	12	77	48	61	55

Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

- First recursive call: Content is 39, 23, 12, 77, 48, 61, 55, and k is 4.

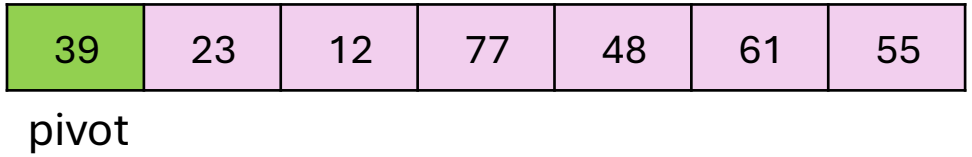
39	23	12	77	48	61	55

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```

Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

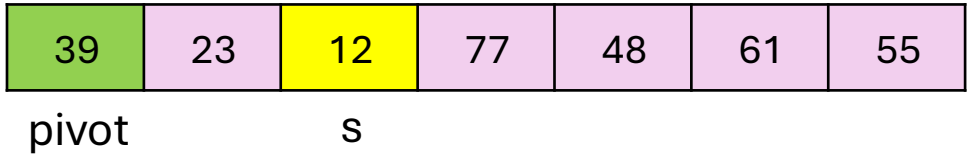
- First recursive call: Content is 39, 23, 12, 77, 48, 61, 55, and k is 4.
- Partitioning proceeds as follows: 39 is the pivot, and a (useless) swap happens for each of the next two elements, as they are smaller than the pivot.



Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

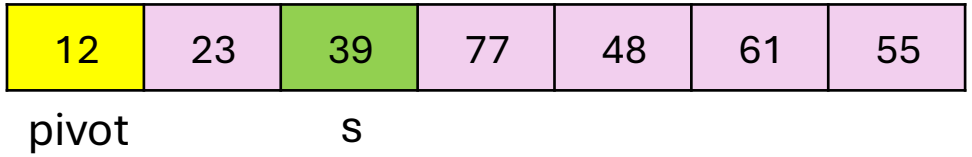
- First recursive call: Content is 39, 23, 12, 77, 48, 61, 55, and k is 4.
- Partitioning proceeds as follows: 39 is the pivot, and a (useless) swap happens for each of the next two elements, as they are smaller than the pivot.
- After that, all elements are greater than the pivot, so we end up with s = 2 (pointing to 12 in 39, 23, 12, 77, 48, 61, 55). Swapping 12 and the pivot, we get 12, 23, 39, 77, 48, 61, 55.



Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

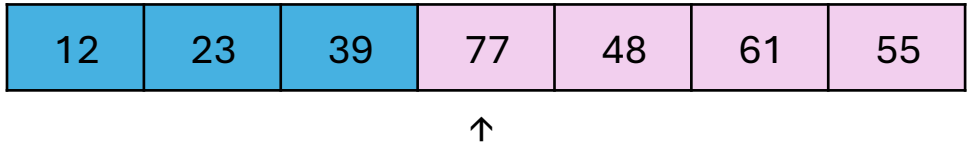
- First recursive call: Content is 39, 23, 12, 77, 48, 61, 55, and k is 4.
- Partitioning proceeds as follows: 39 is the pivot, and a (useless) swap happens for each of the next two elements, as they are smaller than the pivot.
- After that, all elements are greater than the pivot, so we end up with s = 2 (pointing to 12 in 39, 23, 12, 77, 48, 61, 55). Swapping 12 and the pivot, we get 12, 23, 39, 77, 48, 61, 55.



Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

- Since s = 2 < 3 = 0 + 4 – 1 = lo + k – 1, the recursive call focuses on the sequence from index 3 onwards, and the new k in the recursive call is k – (s + 1) = 1. That is, the process repeats, now looking for the smallest element in the sequence 77, 48, 61, 55.

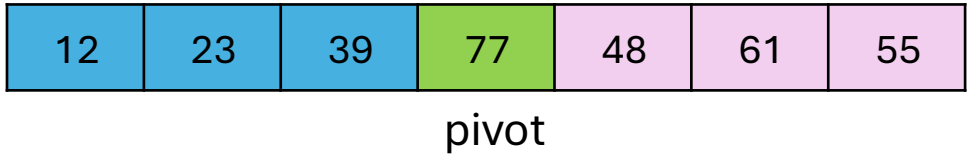



```
function QUICKSELECT(A[·], lo, hi, k)
    s ← LOMUTOPARTITION(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QUICKSELECT(A, lo, s − 1, k)
        else
            QUICKSELECT(A, s + 1, hi, (k − 1) − (s − lo))
```

Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

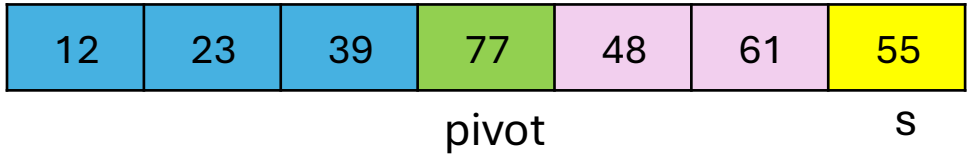
- Since s = 2 < 3 = 0 + 4 − 1 = lo + k − 1, the recursive call focuses on the sequence from index 3 onwards, and the new k in the recursive call is k − (s + 1) = 1. That is, the process repeats, now looking for the smallest element in the sequence 77, 48, 61, 55.
- This time, partitioning proceeds as follows: With 77 being pivot, all other elements are smaller, so three useless swaps take place, changing nothing.



Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

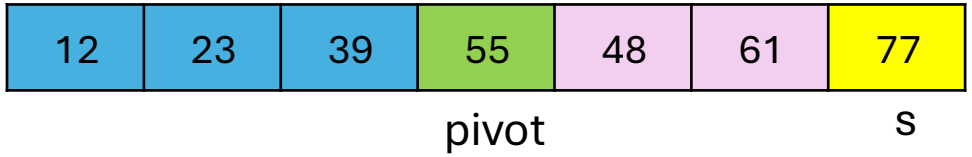
- Since s = 2 < 3 = 0 + 4 − 1 = lo + k − 1, the recursive call focuses on the sequence from index 3 onwards, and the new k in the recursive call is k − (s + 1) = 1. That is, the process repeats, now looking for the smallest element in the sequence 77, 48, 61, 55.
- This time, partitioning proceeds as follows: With 77 being pivot, all other elements are smaller, so three useless swaps take place, changing nothing.
- Finally item 77 is swapped with the item in position s = 6, leaving us with 55, 48, 61, 77.



Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

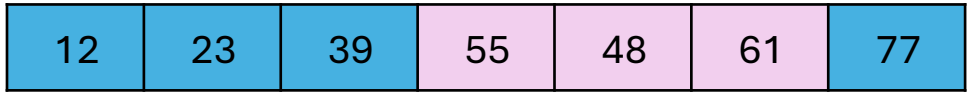
- Since s = 2 < 3 = 0 + 4 − 1 = lo + k − 1, the recursive call focuses on the sequence from index 3 onwards, and the new k in the recursive call is k − (s + 1) = 1. That is, the process repeats, now looking for the smallest element in the sequence 77, 48, 61, 55.
- This time, partitioning proceeds as follows: With 77 being pivot, all other elements are smaller, so three useless swaps take place, changing nothing.
- Finally item 77 is swapped with the item in position s = 6, leaving us with 55, 48, 61, 77.



Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

- Since s > 3+1−1 = 3, the next recursive call focuses on 55, 48, 61 (that is, lo = 3), still with k = 1.

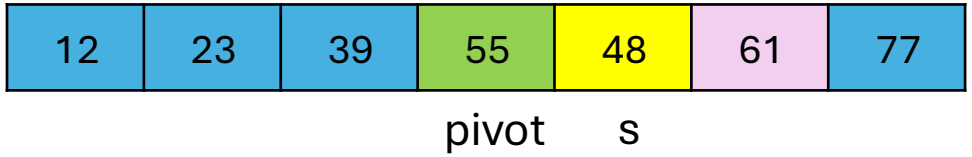



```
function QUICKSELECT(A[·], lo, hi, k)
    s ← LOMUTOPARTITION(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QUICKSELECT(A, lo, s − 1, k)
        else
            QUICKSELECT(A, s + 1, hi, (k − 1) − (s − lo))
```

Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

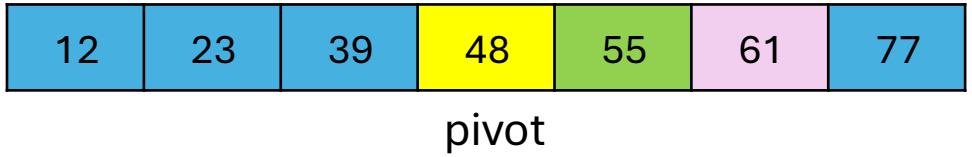
- Since s > 3+1−1 = 3, the next recursive call focuses on 55, 48, 61 (that is, lo = 3), still with k = 1.
- This time partitioning swaps 55 and 48, and returns s = 4.



Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

- Since s > 3+1−1 = 3, the next recursive call focuses on 55, 48, 61 (that is, lo = 3), still with k = 1.
- This time partitioning swaps 55 and 48, and returns s = 4.
- So, a final recursive call happens, focusing very narrowly on item 48 only, with k = 1.

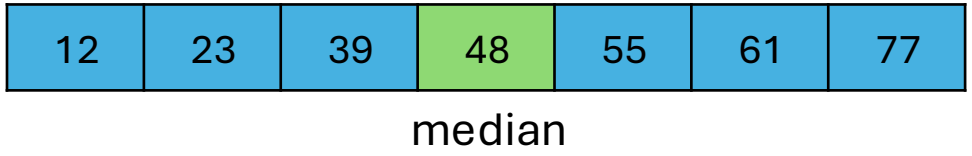



```
function QUICKSELECT(A[·], lo, hi, k)
    s ← LOMUTOPARTITION(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QUICKSELECT(A, lo, s − 1, k)
        else
            QUICKSELECT(A, s + 1, hi, (k − 1) − (s − lo))
```

Tutorial Questions

Problem 1. Trace how QUICKSELECT finds the median of 39, 23, 12, 77, 48, 61, 55.

- Since s > 3+1−1 = 3, the next recursive call focuses on 55, 48, 61 (that is, lo = 3), still with k = 1.
- This time partitioning swaps 55 and 48, and returns s = 4.
- So, a final recursive call happens, focusing very narrowly on item 48 only, with k = 1.
- This call returns 48 as the final result.




```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```

Tutorial Questions

43. We can use QUICKSELECT to find the smallest element of an unsorted array. How does it compare to the more obvious way of solving the problem, namely scanning through the array and maintaining a variable *min* that holds the smallest element found so far?

```
function LomutoPartition(A[·], lo, hi)
    p ← A[lo]
    s ← lo
    for i ← lo + 1 to hi do
        if A[i] < p then
            s ← s + 1
            swap(A[s], A[i])
    swap(A[lo], A[s])
    return s
```

```
function QuickSelect(A[·], lo, hi, k)
    s ← LomutoPartition(A, lo, hi)
    if s − lo = k − 1 then
        return A[s]
    else
        if s − lo > k − 1 then
            QuickSelect(A, lo, s − 1, k)
        else
            QuickSelect(A, s + 1, hi, (k − 1) − (s − lo))
```

Worst case O(n^2)
Average case O(n)

Tutorial Questions

43. We can use QUICKSELECT to find the smallest element of an unsorted array. How does it compare to the more obvious way of solving the problem, namely scanning through the array and maintaining a variable *min* that holds the smallest element found so far?

- The straight-forward way of scanning through the array is better. It will require n − 1 comparisons. QuickSelect will require that number of comparisons just to do the first round of partitioning, and of course one round may not be enough. In fact, in the worst case we end up doing $\Theta(n^2)$ comparisons.

Tutorial Questions

44. Apply mergesort to the list S, O, R, T, X, A, M, P, L, E.

Tip

When dividing an array of odd length n,

- the left sub-array should have length $\lfloor n/2 \rfloor$,
- the right sub-array should have length $\lceil n/2 \rceil$.

```
procedure MERGESORT(A[·], n)                          ▷ Sort A[0]..A[n − 1]
    if n > 1 then
        for i ← 0 to ⌊n/2⌋ − 1 do                     ▷ Copy left half of A to B
            B[i] ← A[i]
        for i ← 0 to ⌈n/2⌉ − 1 do                     ▷ Copy right half of A to C
            C[i] ← A[⌊n/2⌋ + i]
        MERGESORT(B, ⌊n/2⌋)                           ▷ Sort B
        MERGESORT(C, ⌈n/2⌉)                           ▷ Sort C
        MERGE(B, ⌊n/2⌋, C, ⌈n/2⌉, A)                 ▷ Merge B and C into A
```

```
procedure MERGE(B[·], p, C[·], q, A[·])
    i ← 0; j ← 0; k ← 0
    while i < p and j < q do
        if B[i] ≤ C[j] then
            A[k] ← B[i]
            i ← i + 1
        else
            A[k] ← C[j]
            j ← j + 1
        k ← k + 1
    if i = p then
        copy C[j]..C[q − 1] to A[k]..A[p + q − 1]      ▷ (a for loop)
    else
        copy B[i]..B[p − 1] to A[k]..A[p + q − 1]      ▷ (a for loop)
```

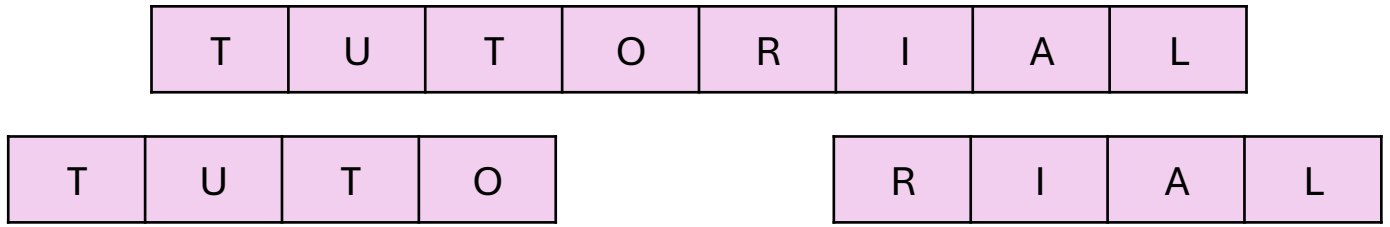
Worst case

Space complexity

MergeSort example

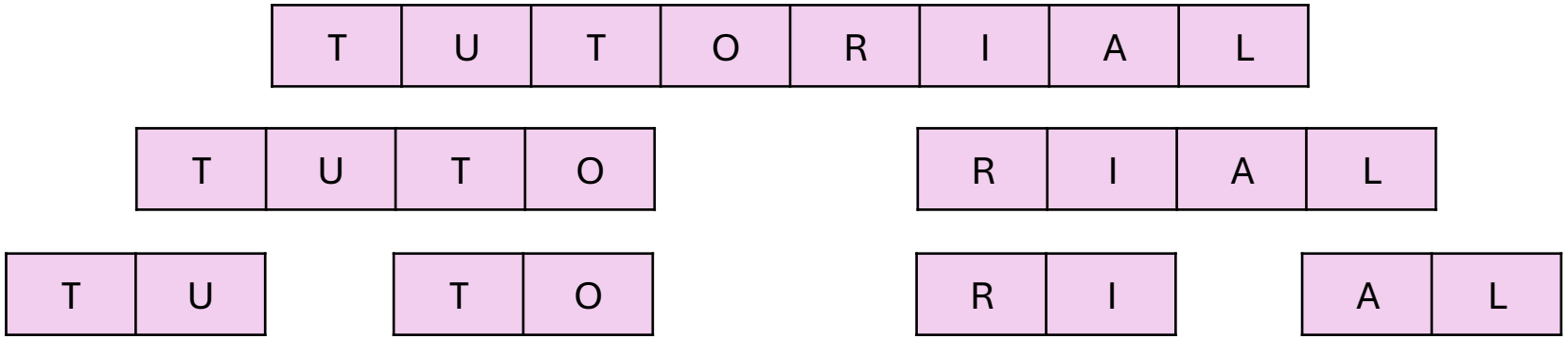


MergeSort example



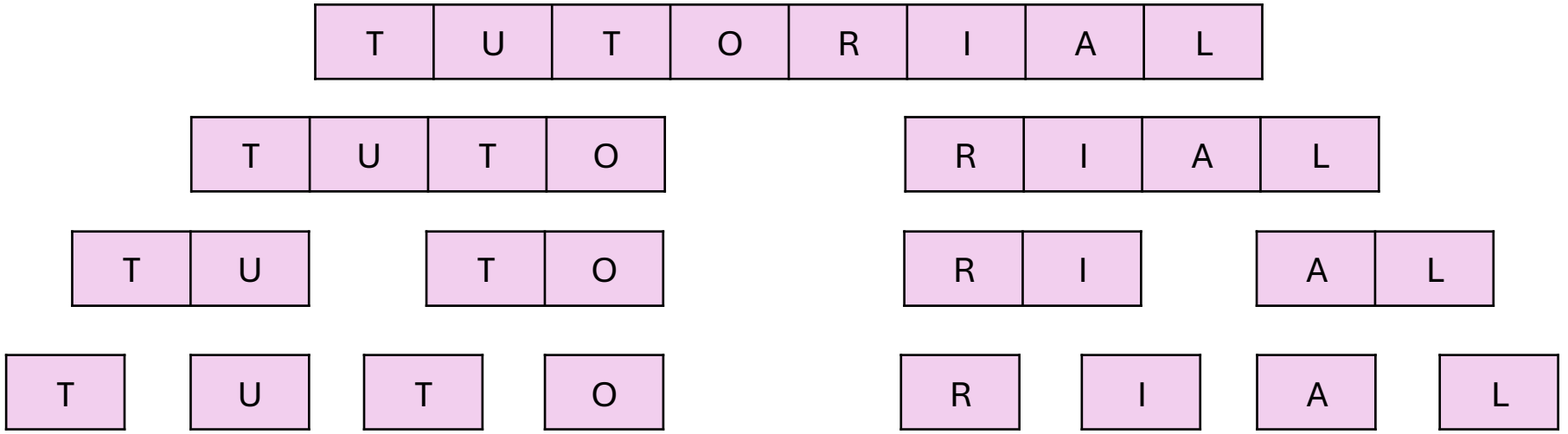
Recursively divide array(s) into sub-arrays of length $\left\lfloor \frac{n}{2} \right\rfloor$ and $\left\lceil \frac{n}{2} \right\rceil$.

MergeSort example



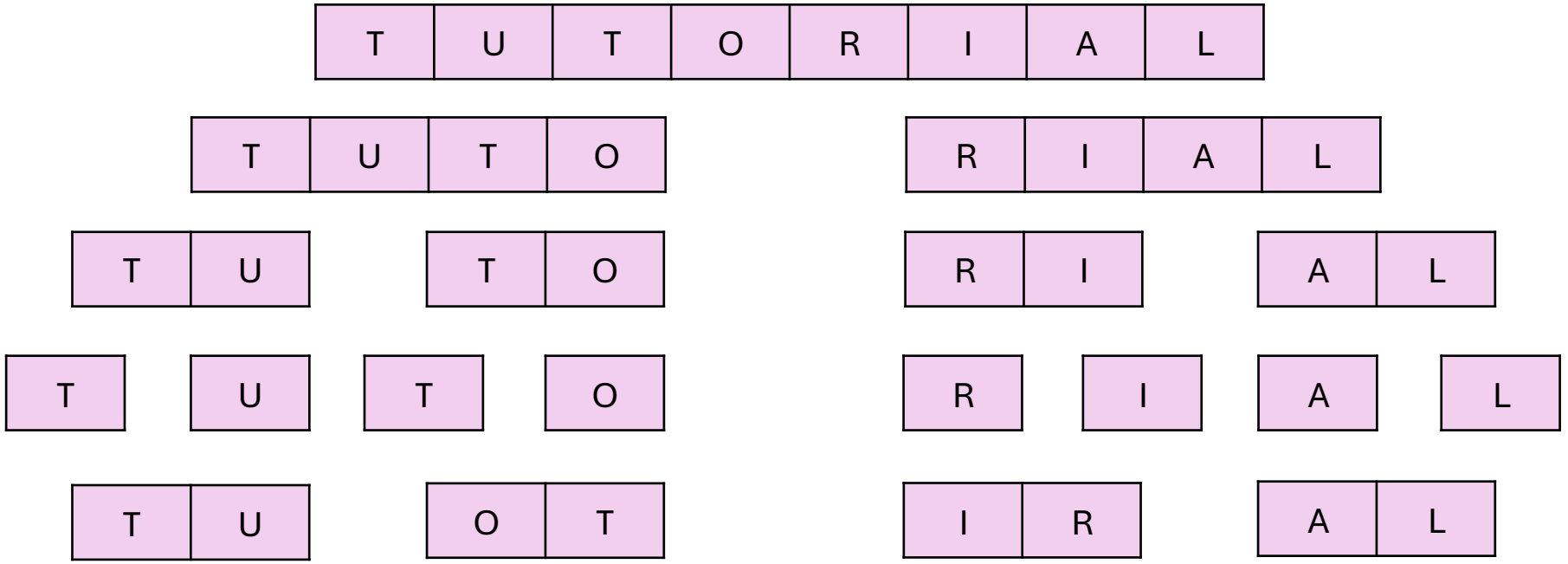
Recursively divide array(s) into sub-arrays of length $\left\lfloor \frac{n}{2} \right\rfloor$ and $\left\lceil \frac{n}{2} \right\rceil$.

MergeSort example



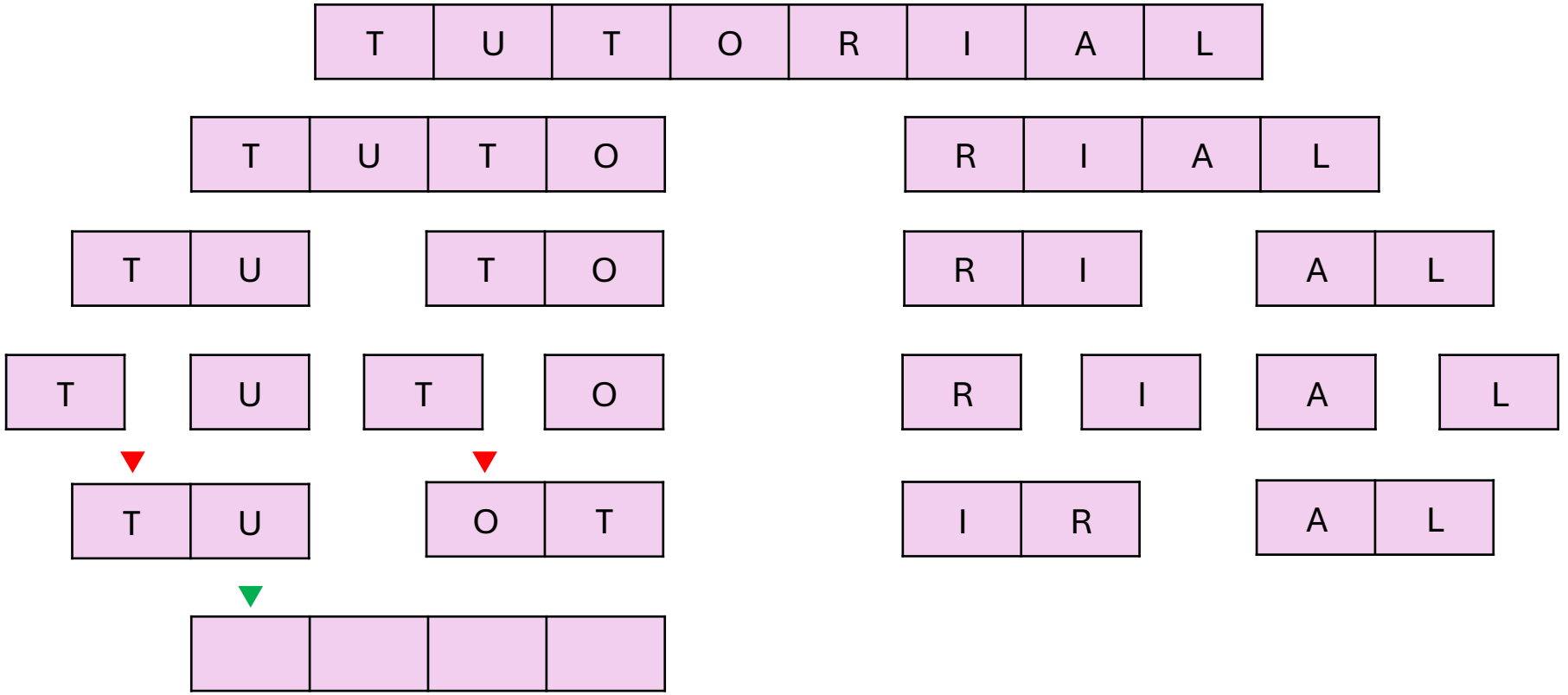
Recursively divide array(s) into sub-arrays of length $\left\lfloor \frac{n}{2} \right\rfloor$ and $\left\lceil \frac{n}{2} \right\rceil$.

MergeSort example



Merge adjacent sub-arrays to get a sorted array.

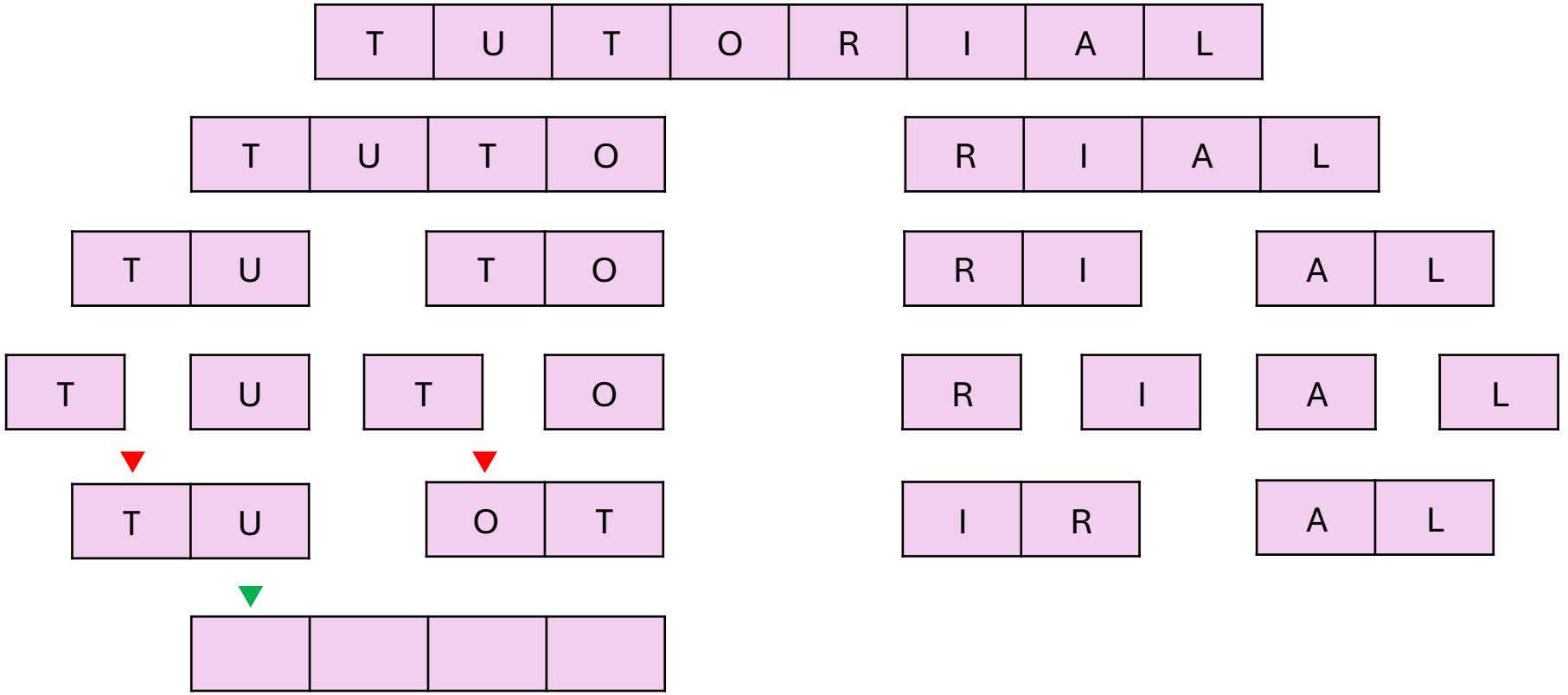
MergeSort example



Merge adjacent sub-arrays to get a sorted array.

Tip: every time you insert a symbol during a ‘merge’ step, move up the pointer on the symbol you just inserted.

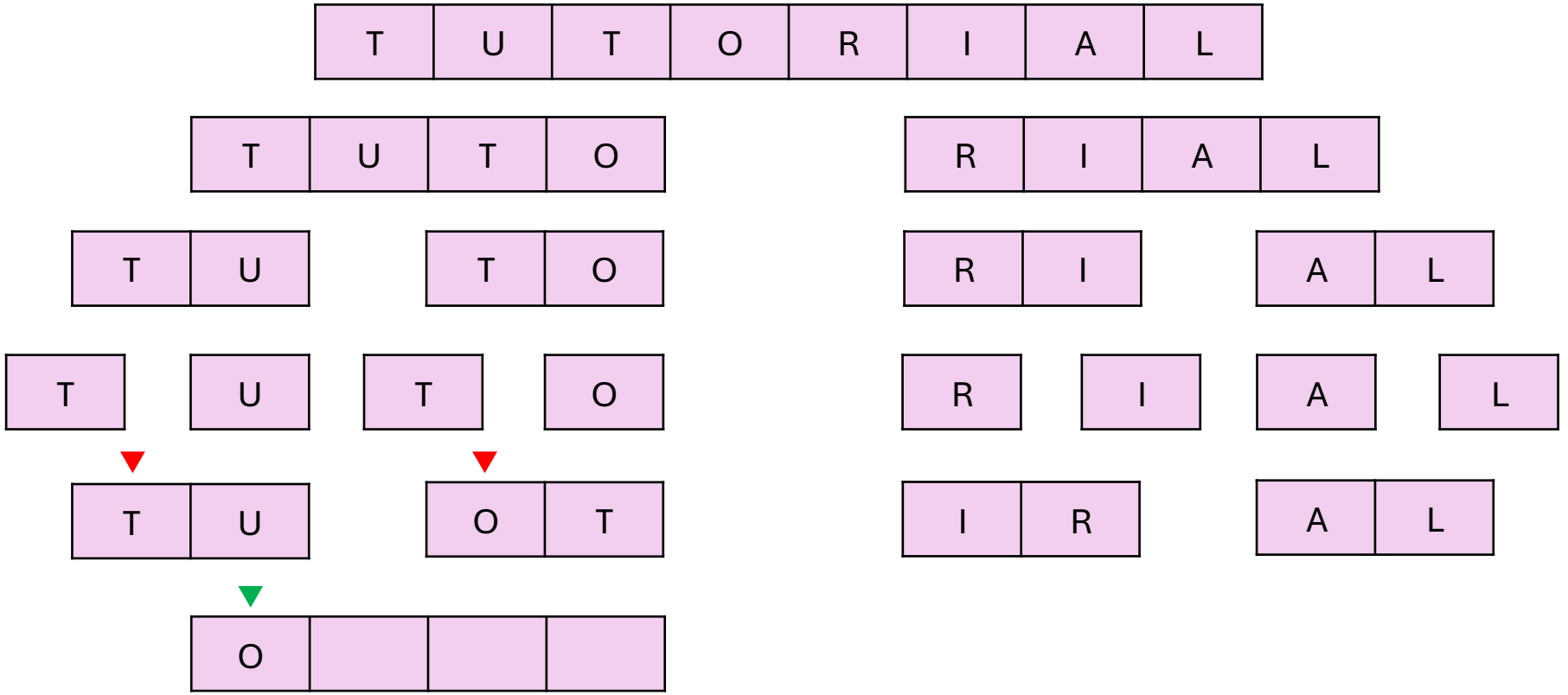
MergeSort example



T > O, so insert O and move up the pointer in the right sub-array.

Merge adjacent sub-arrays to get a sorted array.

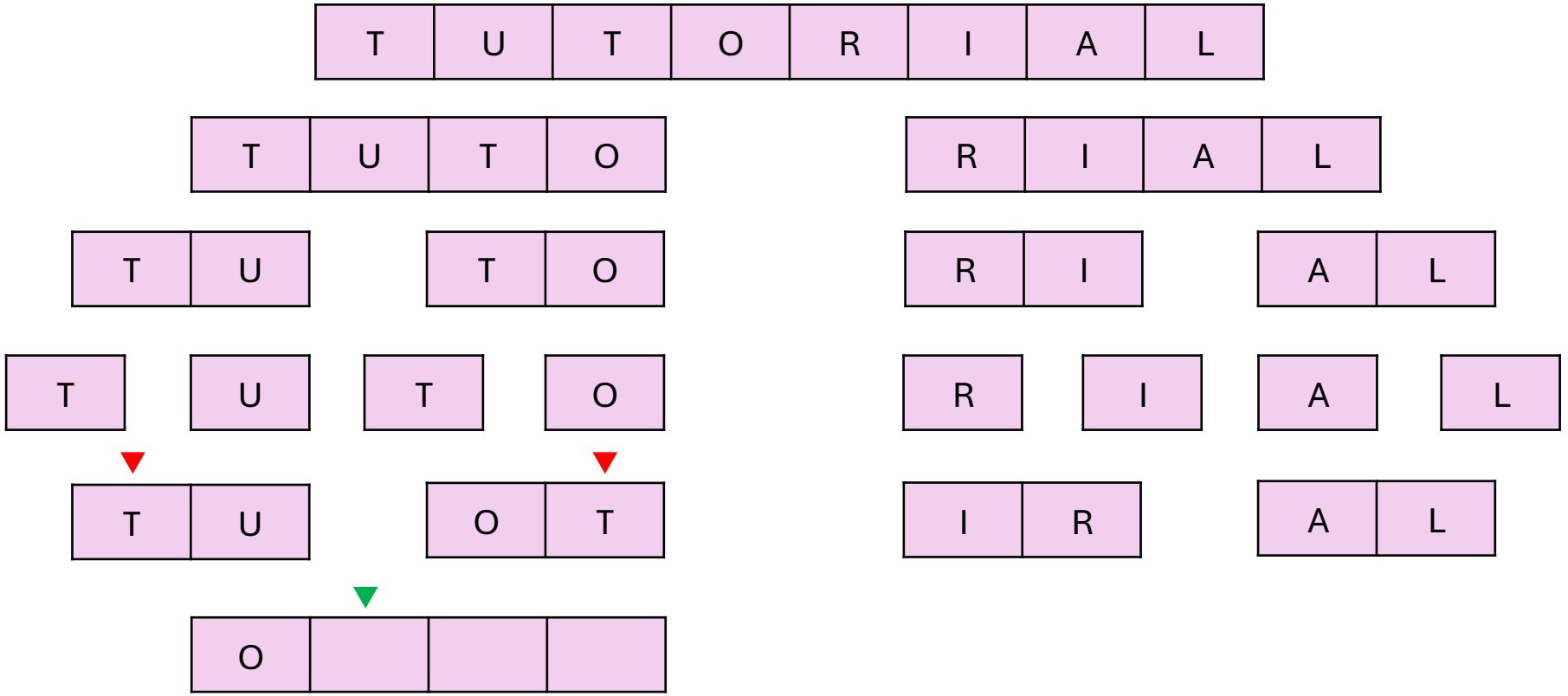
MergeSort example



T > O, so insert O and move up the pointer in the right sub-array.

Merge adjacent sub-arrays to get a sorted array.

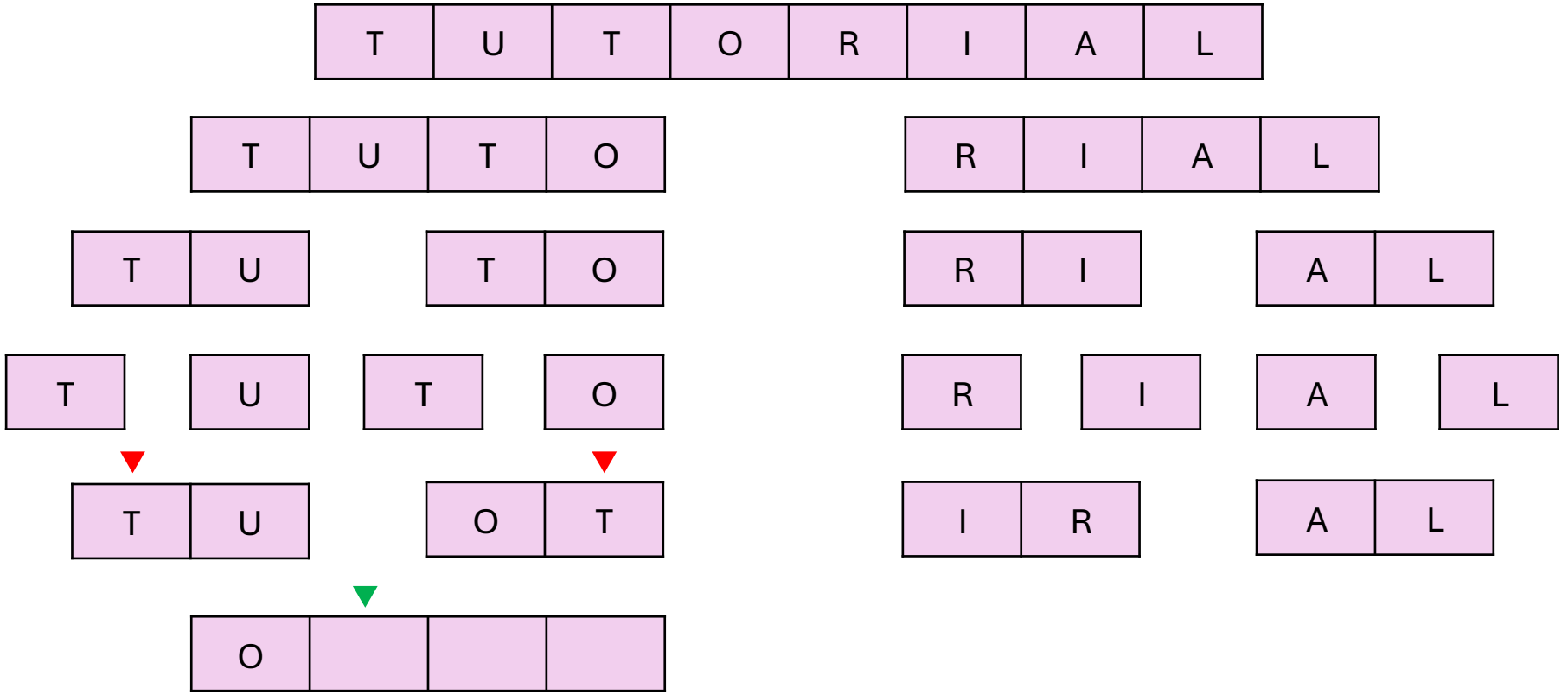
MergeSort example



T > O, so insert O and move up the pointer in the right sub-array.

Merge adjacent sub-arrays to get a sorted array.

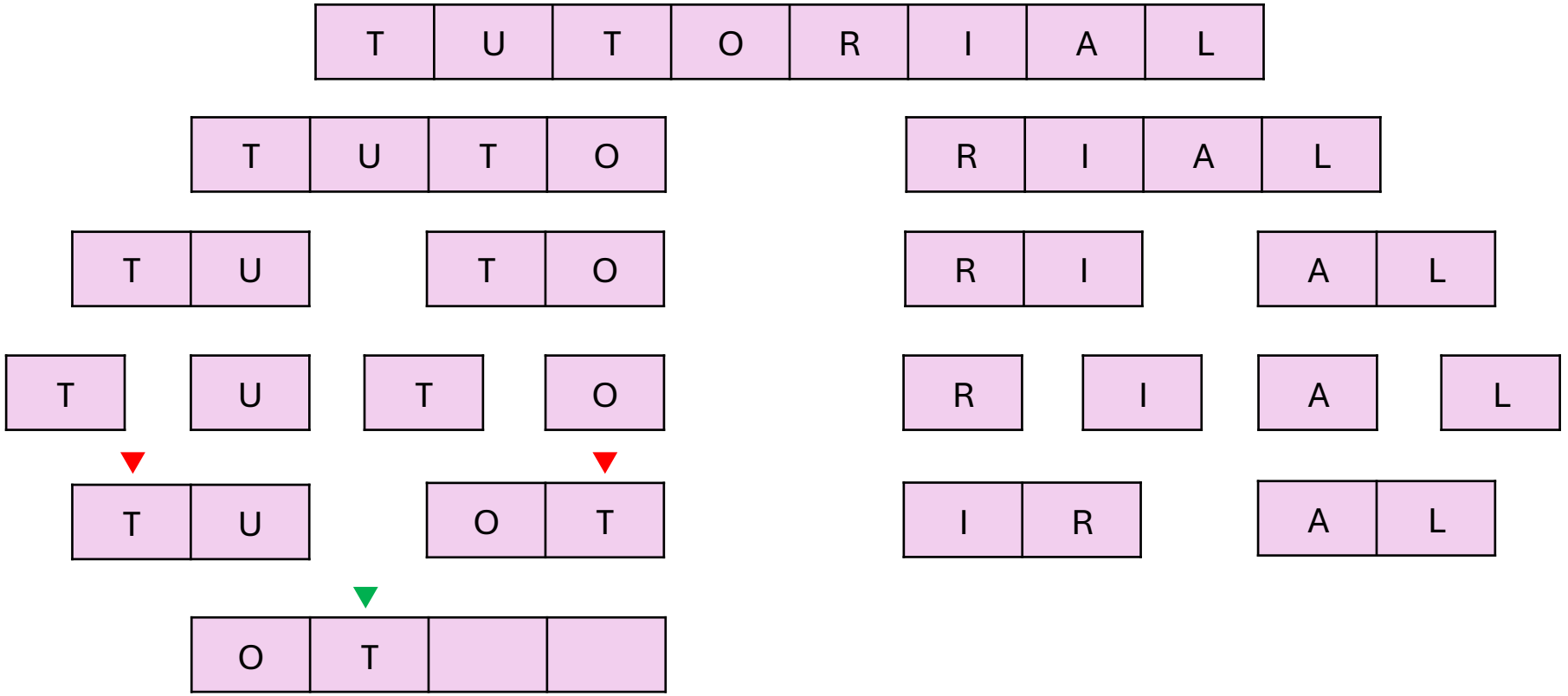
MergeSort example



T ≤ T, so insert (the left) T and move up the pointer in the left sub-array.

Merge adjacent sub-arrays to get a sorted array.

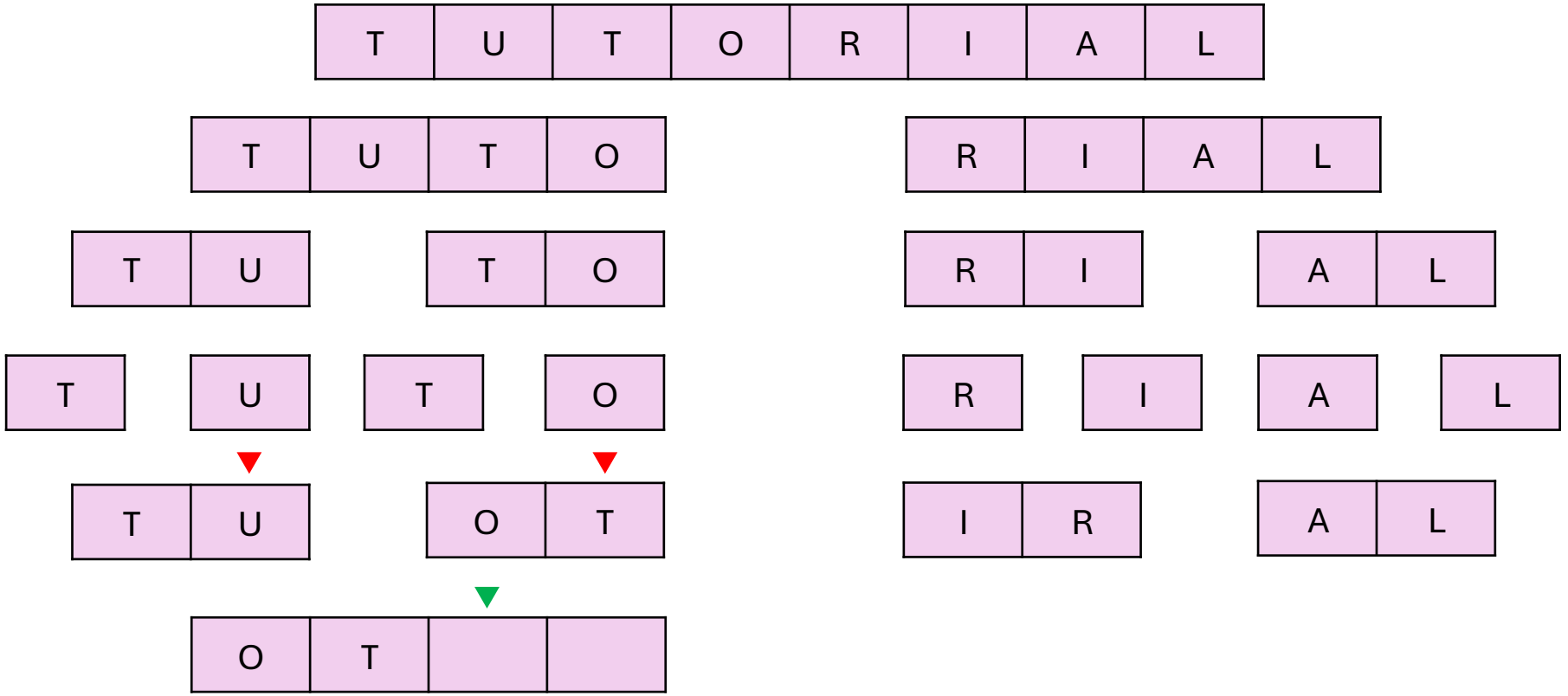
MergeSort example



T ≤ T, so insert (the left) T and move up the pointer in the left sub-array.

Merge adjacent sub-arrays to get a sorted array.

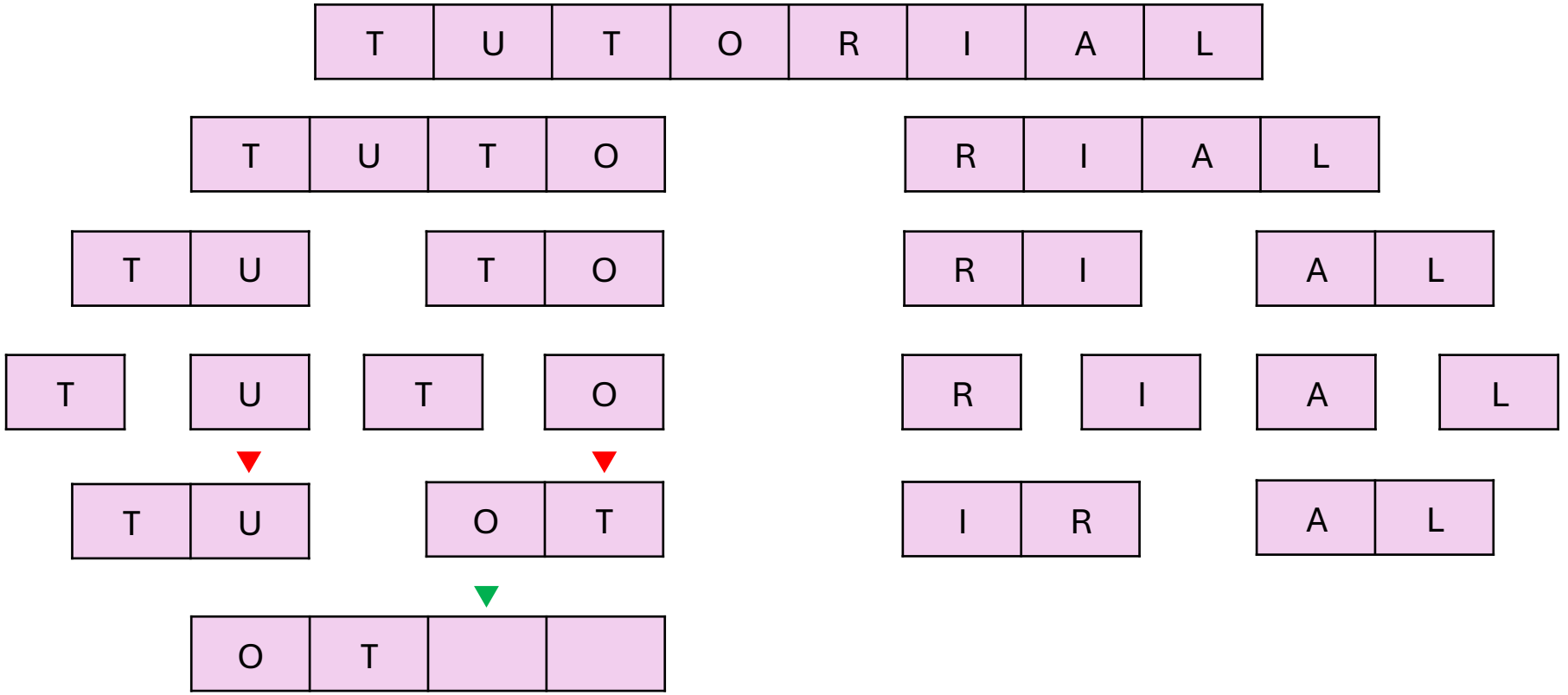
MergeSort example



T ≤ T, so insert (the left) T and move up the pointer in the left sub-array.

Merge adjacent sub-arrays to get a sorted array.

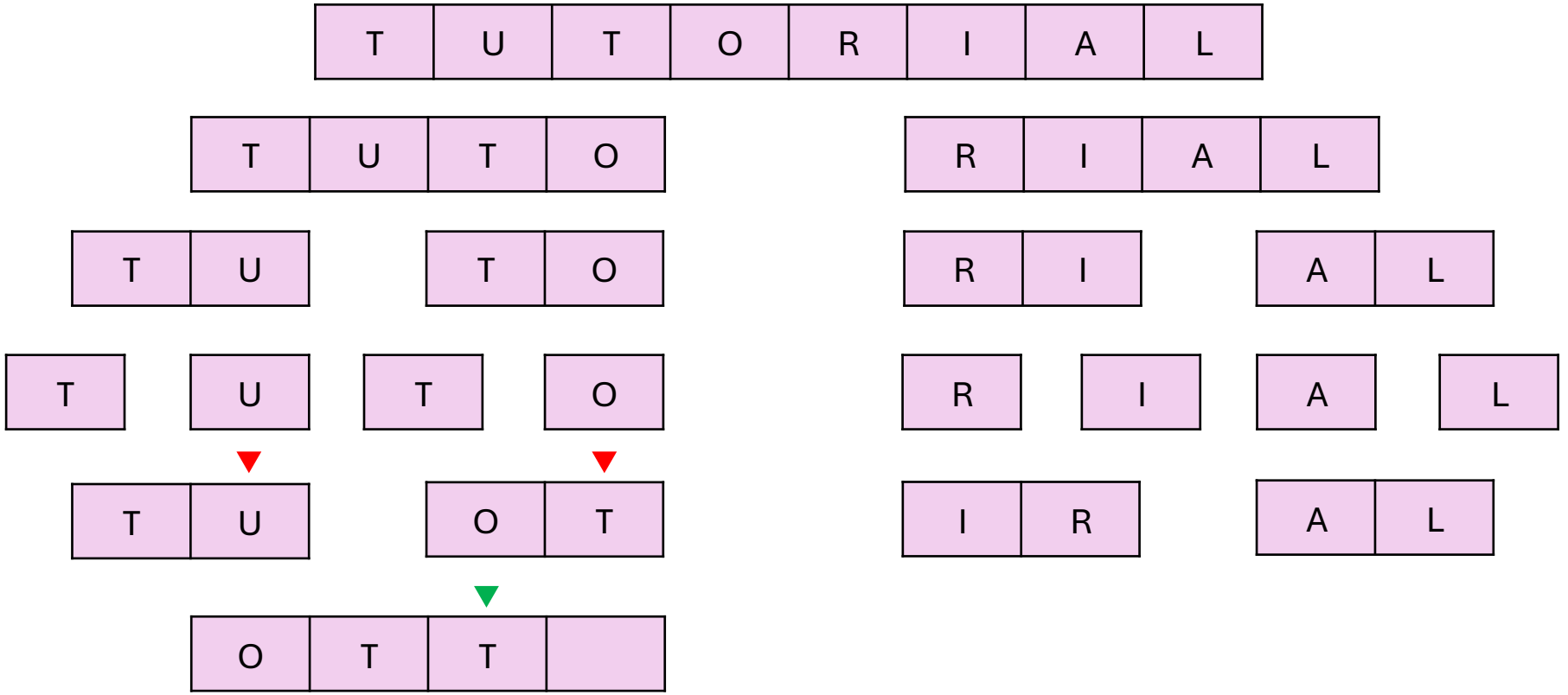
MergeSort example



U > T, so insert T and move up the pointer in the right sub-array.

Merge adjacent sub-arrays to get a sorted array.

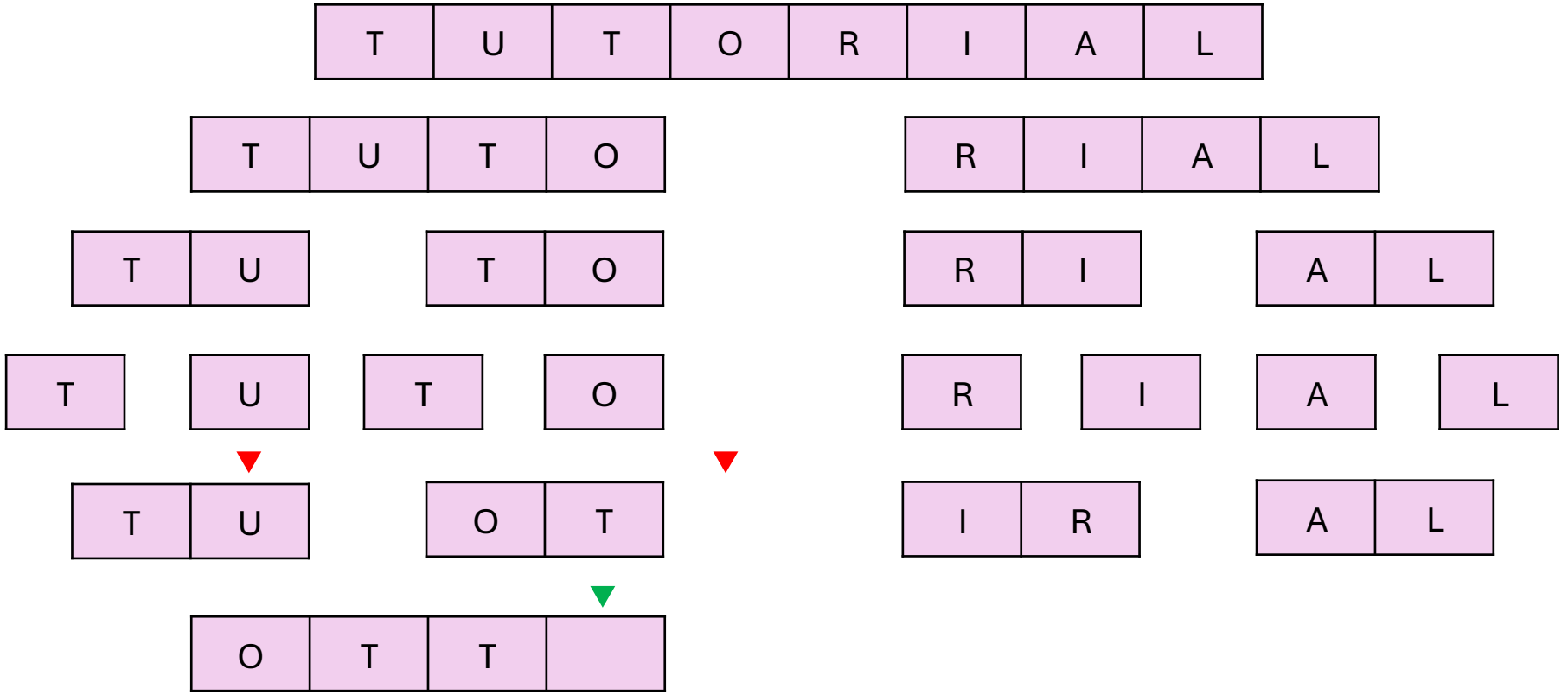
MergeSort example



U > T, so insert T and move up the pointer in the right sub-array.

Merge adjacent sub-arrays to get a sorted array.

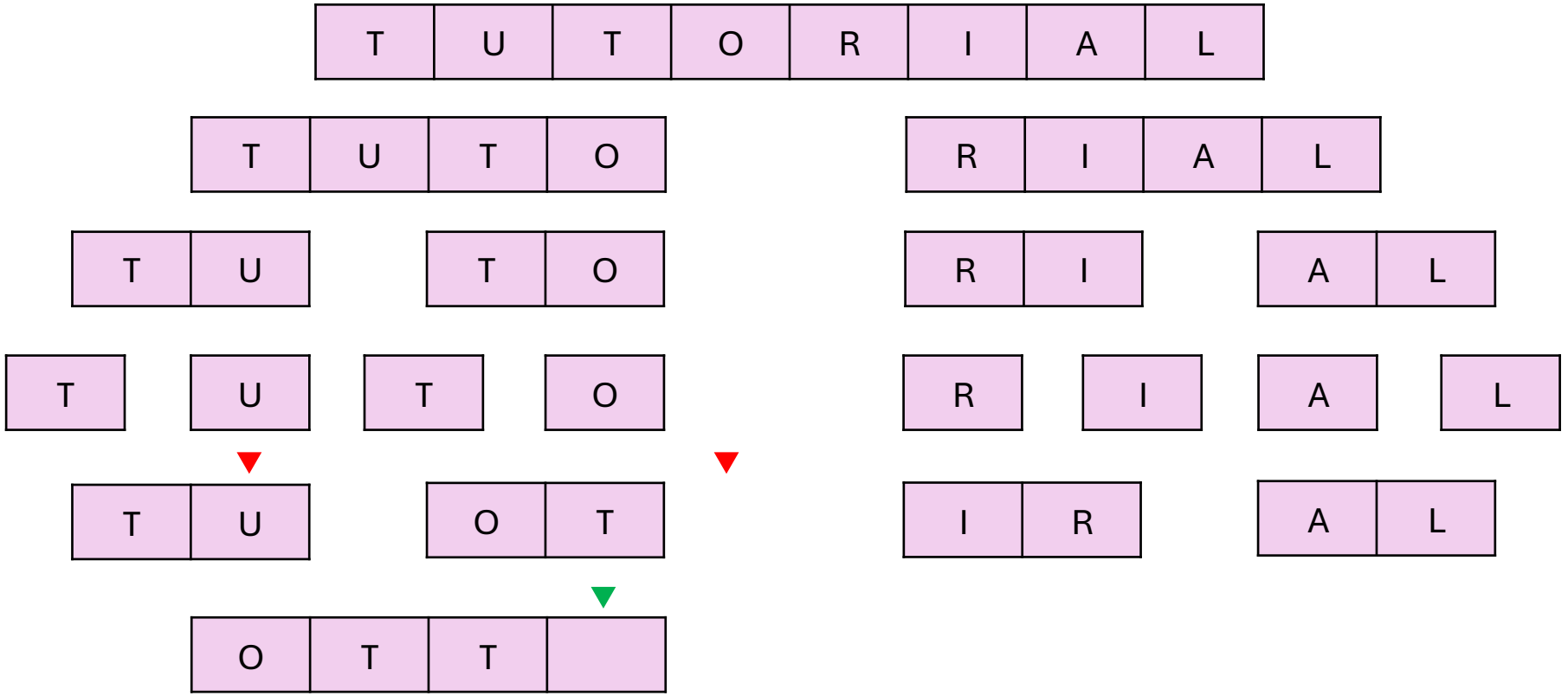
MergeSort example



U > T, so insert T and move up the pointer in the right sub-array.

Merge adjacent sub-arrays to get a sorted array.

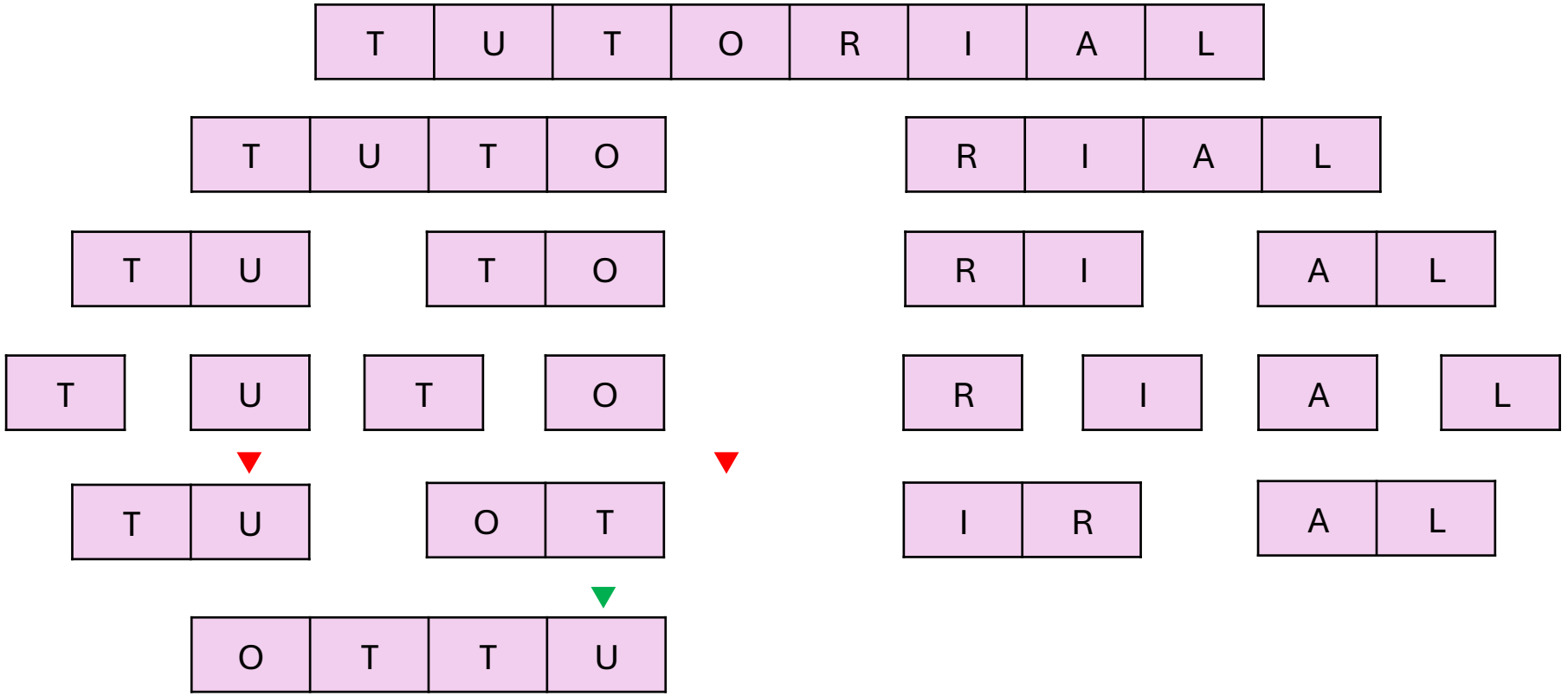
MergeSort example



The right sub-array has no remaining letters, so insert the remaining letters from the left sub-array.

Merge adjacent sub-arrays to get a sorted array.

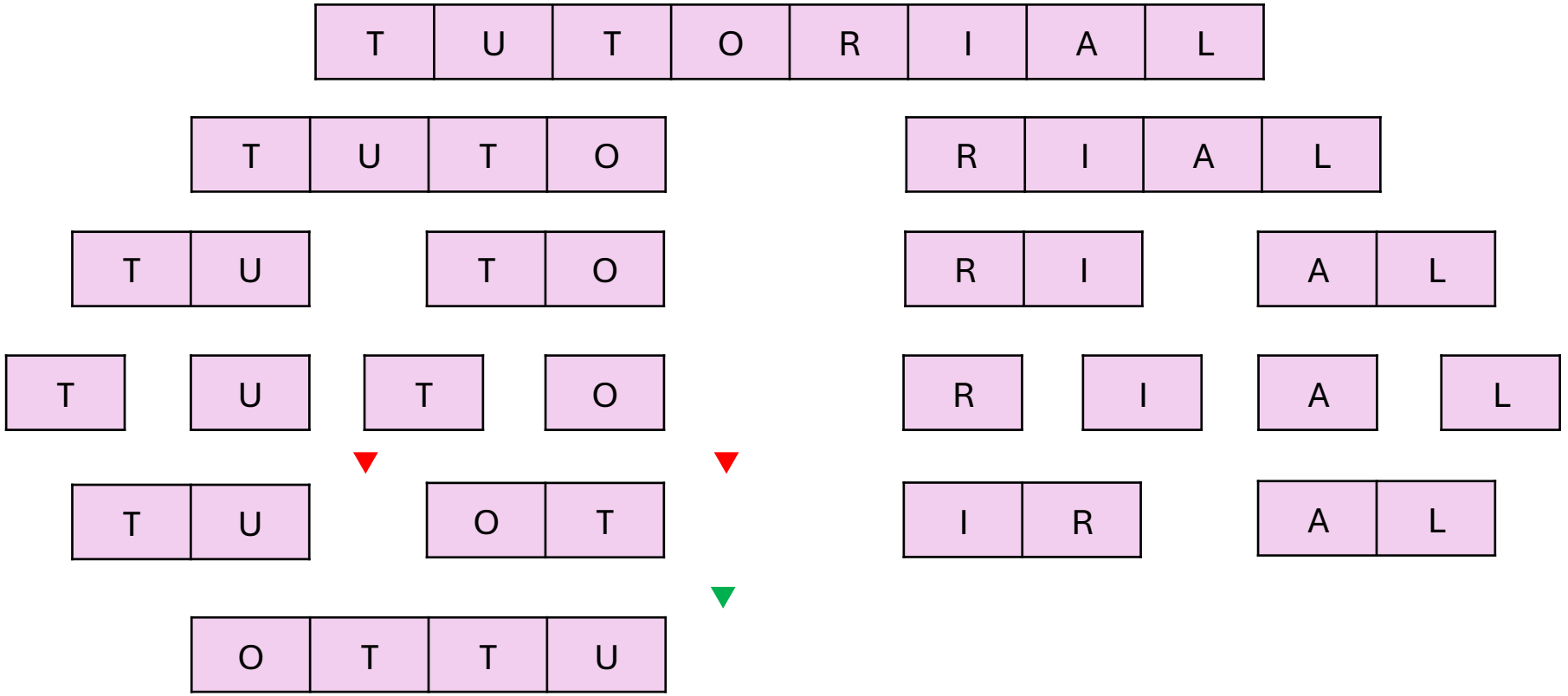
MergeSort example



The right sub-array has no remaining letters, so insert the remaining letters from the left sub-array.

Merge adjacent sub-arrays to get a sorted array.

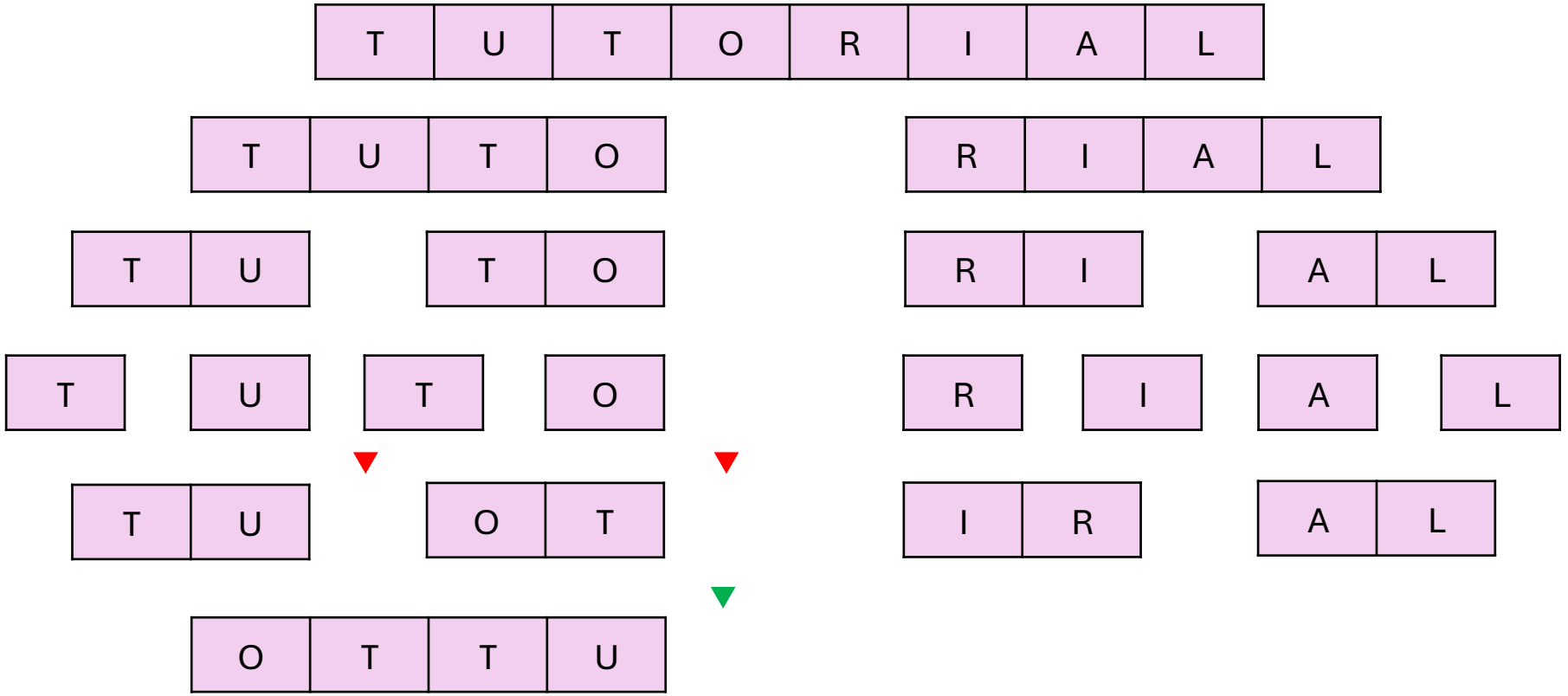
MergeSort example



The right sub-array has no remaining letters, so insert the remaining letters from the left sub-array.

Merge adjacent sub-arrays to get a sorted array.

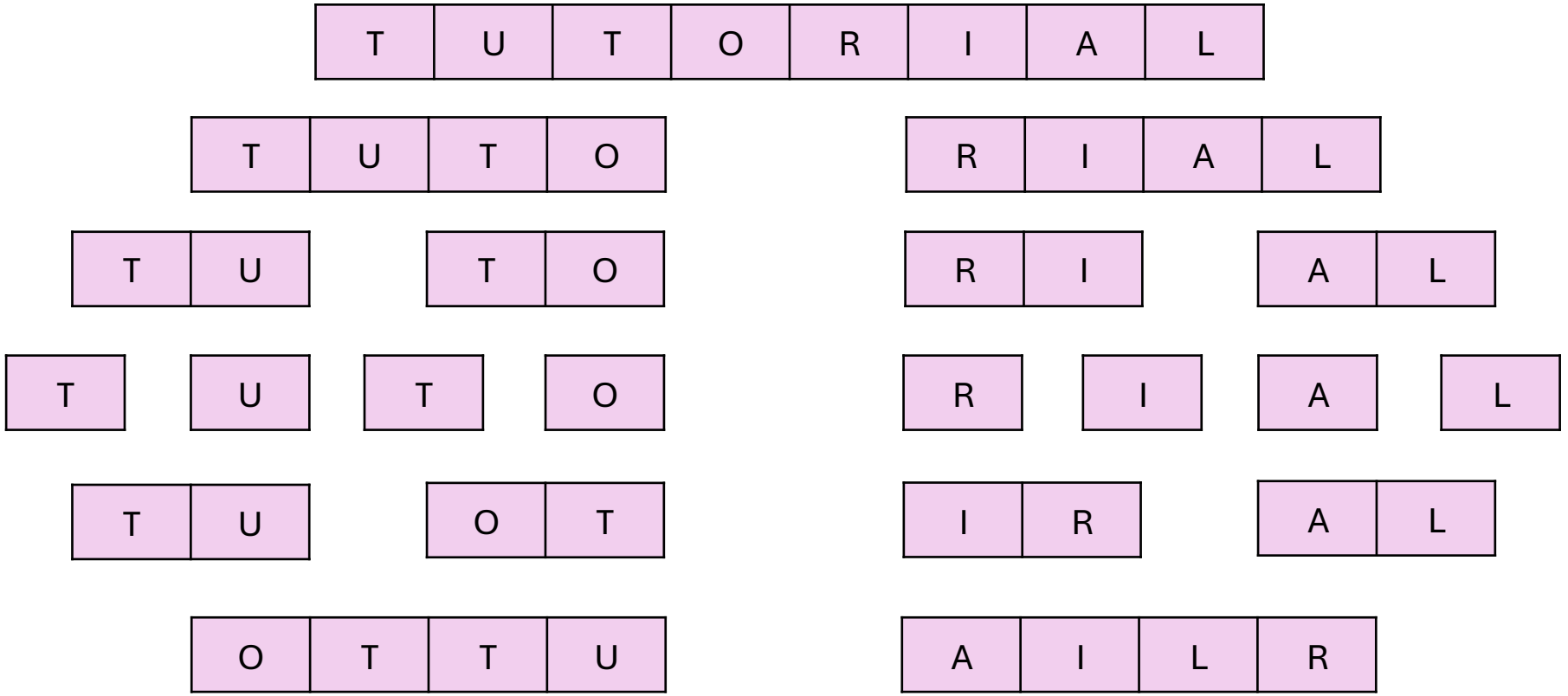
MergeSort example



There are no remaining letters to check for this sub-array.

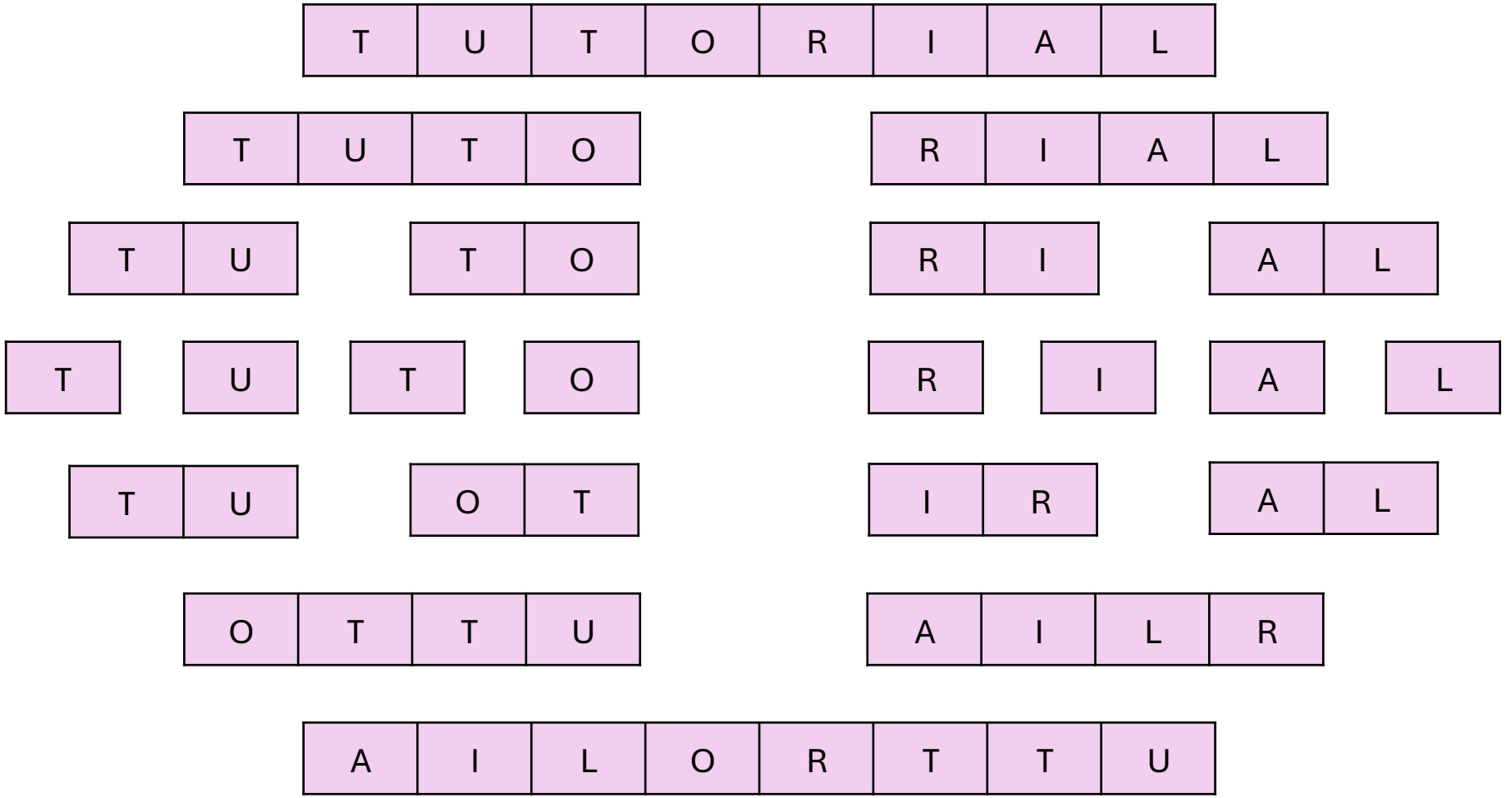
Merge adjacent sub-arrays to get a sorted array.

MergeSort example



Merge adjacent sub-arrays to get a sorted array.

MergeSort example



Merge adjacent sub-arrays to get a sorted array.

45. Apply quicksort (with Hoare partitioning) to the list S, O, R, T, X, A, M, P, L, E.

```
procedure QUICKSORT(A[·], lo, hi)
    if lo < hi then
        s ← PARTITION(A, lo, hi)
        QUICKSORT(A, lo, s − 1)
        QUICKSORT(A, s + 1, hi)
```

```
function PARTITION(A[·], lo, hi)
    p ← A[lo]; i ← lo; j ← hi
    repeat
        while i < hi and A[i] ≤ p do i ← i + 1
        while j ≥ lo and A[j] > p do j ← j − 1
        swap(A[i], A[j])
    until i ≥ j
    swap(A[i], A[j])
    swap(A[lo], A[j])
    return j
```

A:

8	3	9	41
0	1	2	3

QuickSort example

pivot

T	U	T	O	R	I	A	L

T	L	T	O	R	I	A	U

T	L	T	O	R	I	U	A

A	L	T	O	R	I	T	U

Return j = 6

```
function PARTITION(A[·], lo, hi)
    p ← A[lo]; i ← lo; j ← hi
    repeat
        while i < hi and A[i] ≤ p do i ← i + 1
        while j ≥ lo and A[j] > p do j ← j − 1
        swap(A[i], A[j])
    until i ≥ j
    swap(A[i], A[j])
    swap(A[lo], A[j])
    return j
```

A:

8	3	9	41
0	1	2	3

```
procedure QUICKSORT(A[·], lo, hi)
    if lo < hi then
        s ← PARTITION(A, lo, hi)
        QUICKSORT(A, lo, s − 1)
        QUICKSORT(A, s + 1, hi)
```

47. When analysing quicksort in the lecture, we noticed that an already sorted array is a worst-case input. Is that still true if we use median-of three pivot selection?

```
function PARTITION(A[·], lo, hi)
    p ← A[lo]; i ← lo; j ← hi
    repeat
        while i < hi and A[i] ≤ p do i ← i + 1
        while j ≥ lo and A[j] > p do j ← j − 1
        swap(A[i], A[j])
    until i ≥ j
    swap(A[i], A[j])
    swap(A[lo], A[j])
    return j
```

```
procedure QUICKSORT(A[·], lo, hi)
    if lo < hi then
        s ← PARTITION(A, lo, hi)
        QUICKSORT(A, lo, s − 1)
        QUICKSORT(A, s + 1, hi)
```

A:

8	3	9	41
0	1	2	3

Worst case

Average case $T(n)=2T(n/2)+O(n)$

47. When analysing quicksort in the lecture, we noticed that an already sorted array is a worst-case input. Is that still true if we use median-of three pivot selection?

Answer: This is no longer a worst case; in fact it becomes a best case! In this case the median-of-three is in fact the array's median. Hence each of the two recursive calls will be given an array of length at most $n/2$, where n is the length of the whole array. And the arrays passed to the recursive calls are again already-sorted, so the phenomenon is invariant throughout the calls.

Tutorial Questions

48. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

```
procedure MERGESORT(A[·], n)                          ▷ Sort A[0]..A[n − 1]
    if n > 1 then
        for i ← 0 to ⌊n/2⌋ − 1 do                     ▷ Copy left half of A to B
            B[i] ← A[i]
        for i ← 0 to ⌈n/2⌉ − 1 do                     ▷ Copy right half of A to C
            C[i] ← A[⌊n/2⌋ + i]
        MERGESORT(B, ⌊n/2⌋)                           ▷ Sort B
        MERGESORT(C, ⌈n/2⌉)                           ▷ Sort C
        MERGE(B, ⌊n/2⌋, C, ⌈n/2⌉, A)                 ▷ Merge B and C into A
```

```
procedure MERGE(B[·], p, C[·], q, A[·])
    i ← 0; j ← 0; k ← 0
    while i < p and j < q do
        if B[i] ≤ C[j] then
            A[k] ← B[i]
            i ← i + 1
        else
            A[k] ← C[j]
            j ← j + 1
        k ← k + 1
    if i = p then
        copy C[j]..C[q − 1] to A[k]..A[p + q − 1]      ▷ (a for loop)
    else
        copy B[i]..B[p − 1] to A[k]..A[p + q − 1]      ▷ (a for loop)
```

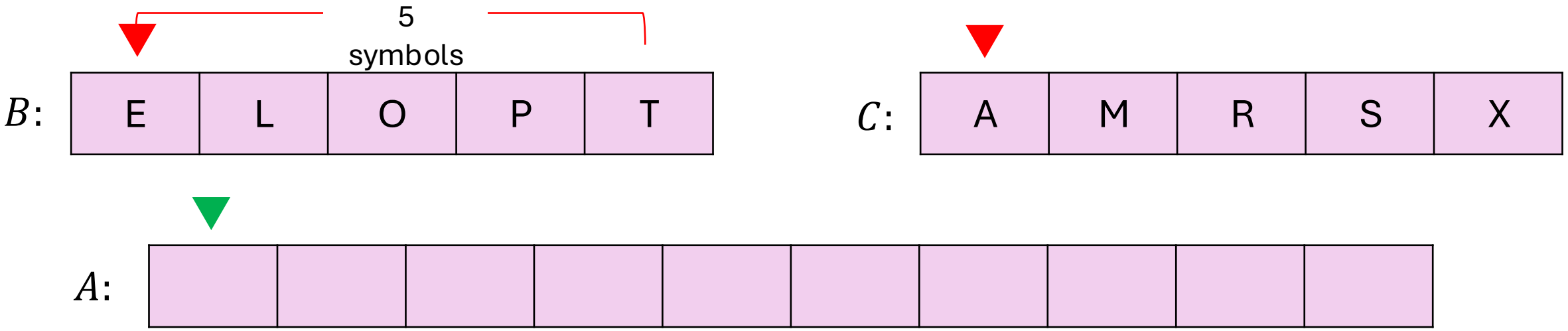
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 0

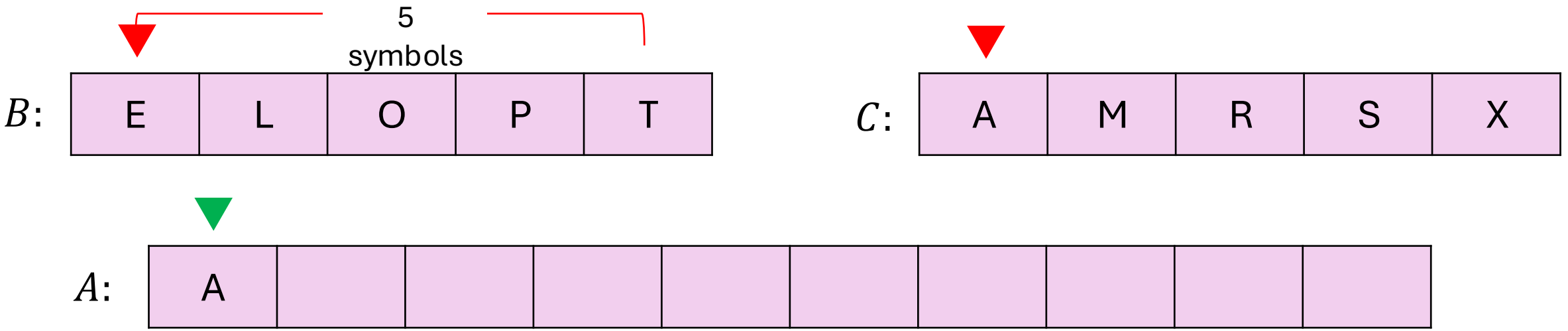
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 0+5

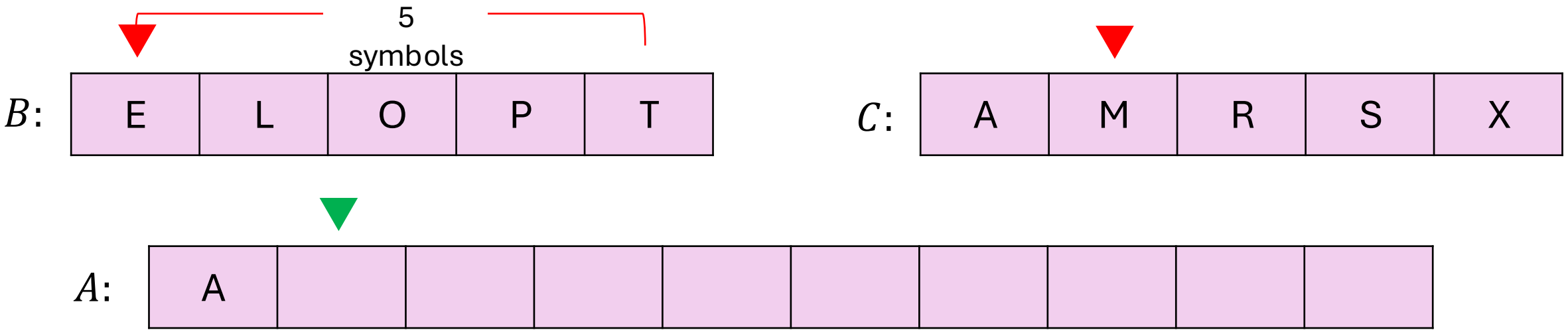
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 5

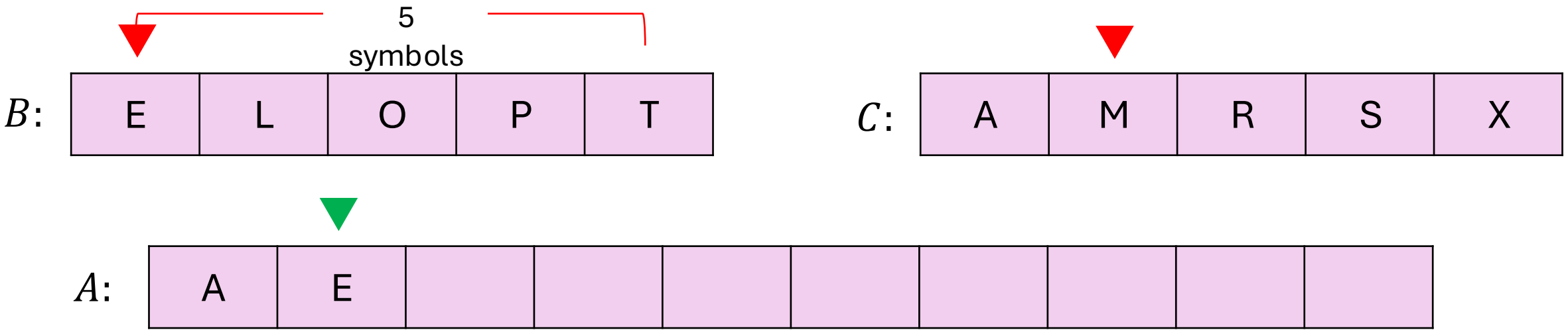
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 5

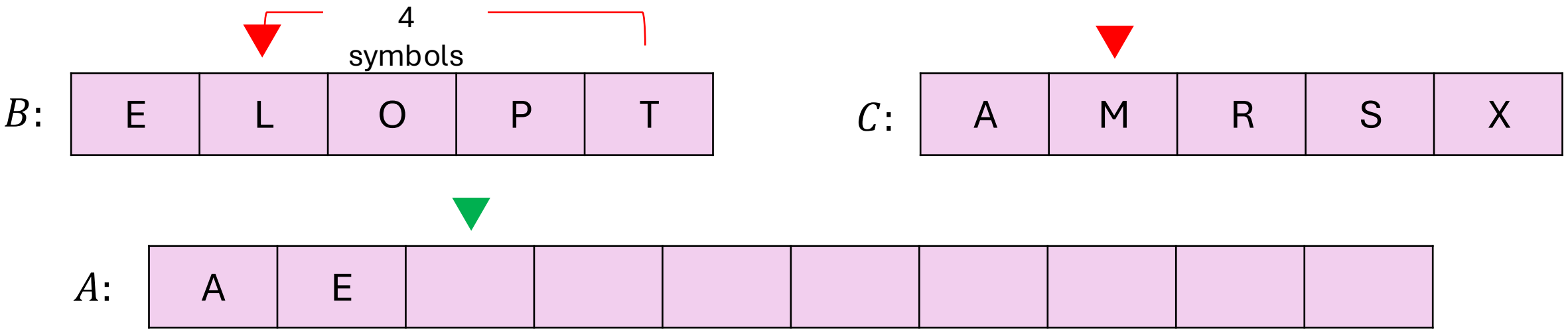
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 5

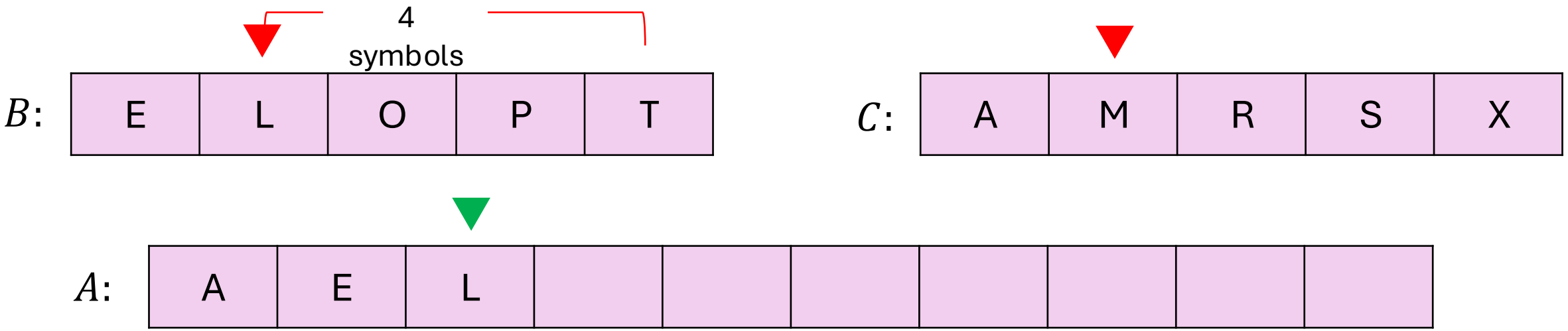
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 5

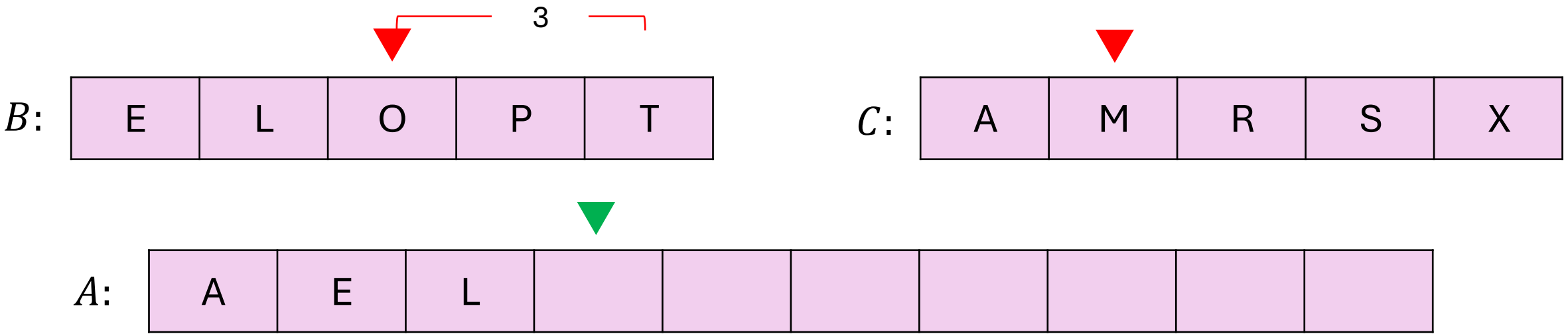
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 5

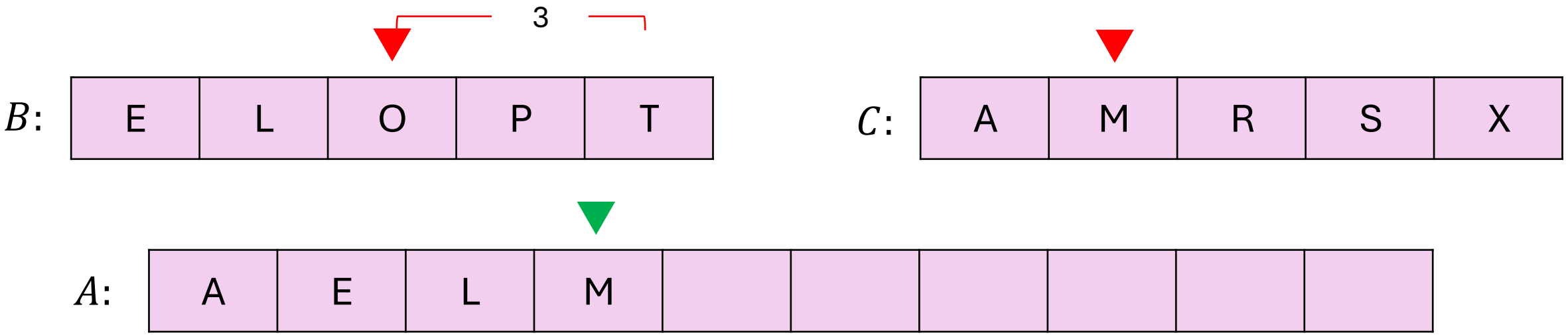
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 5+3

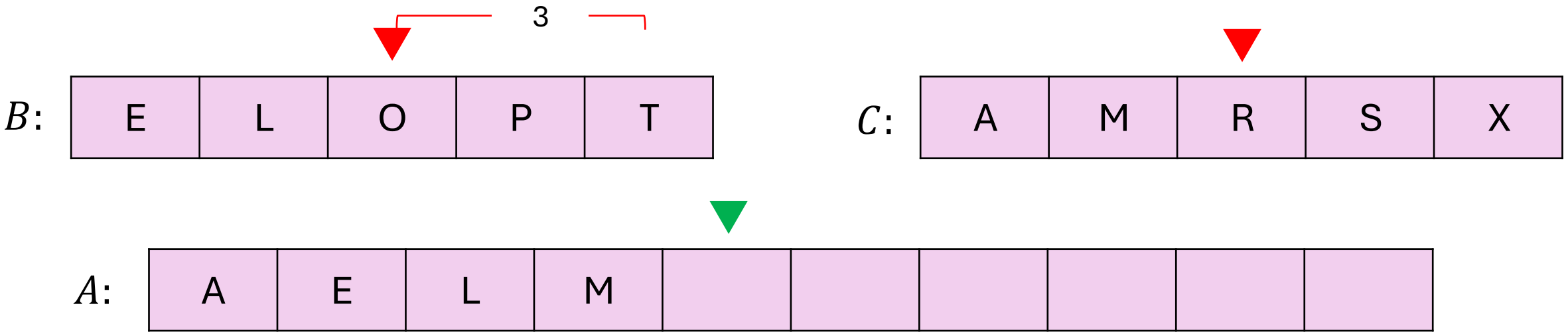
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 8

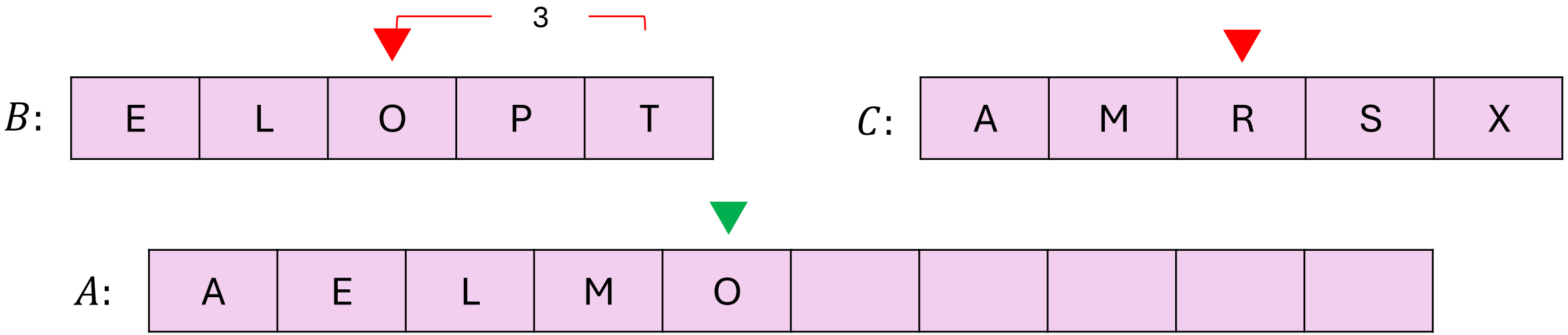
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 8

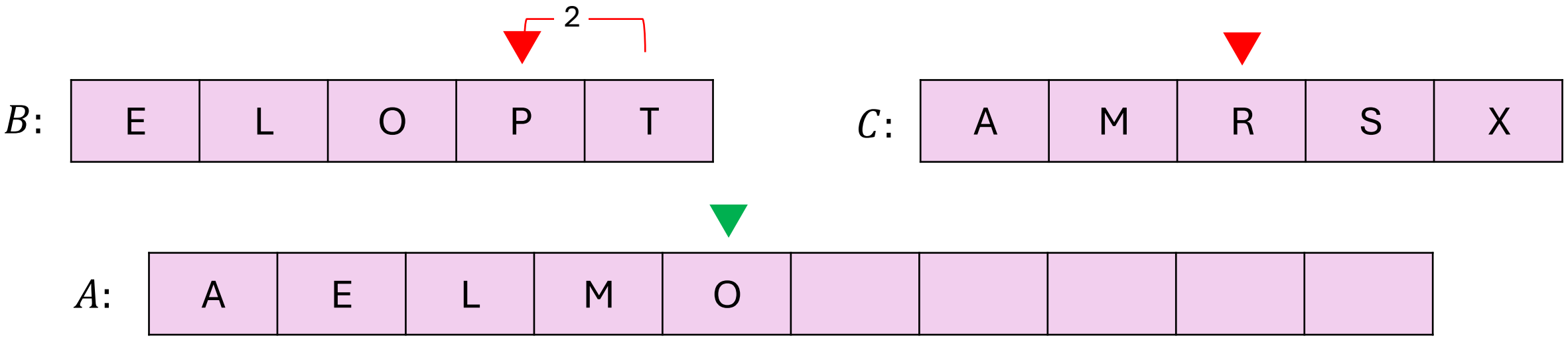
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 8

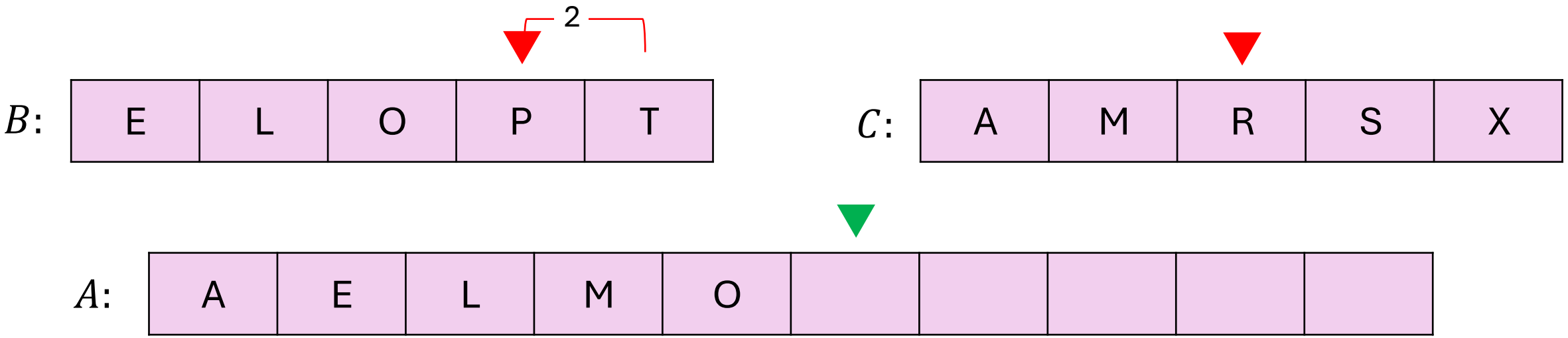
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 8

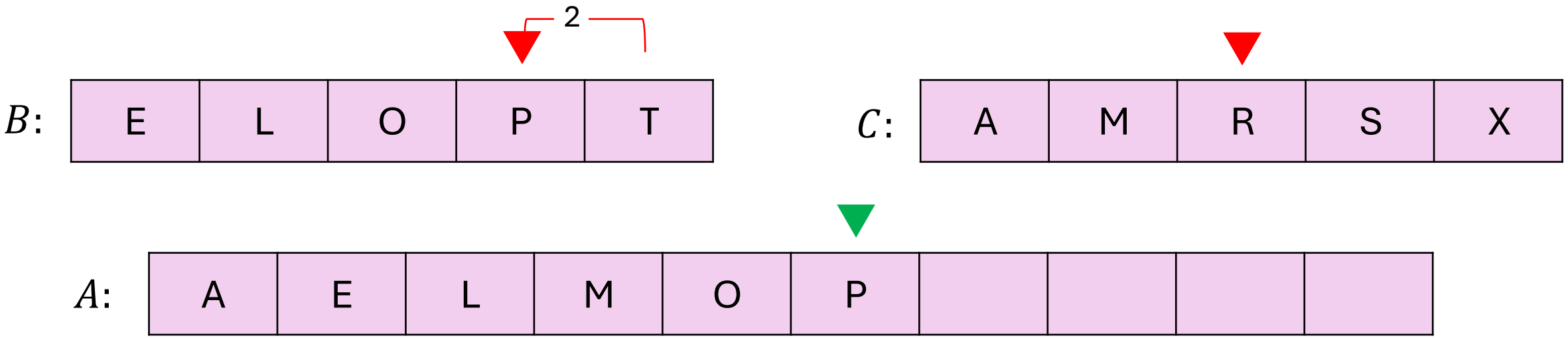
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 8

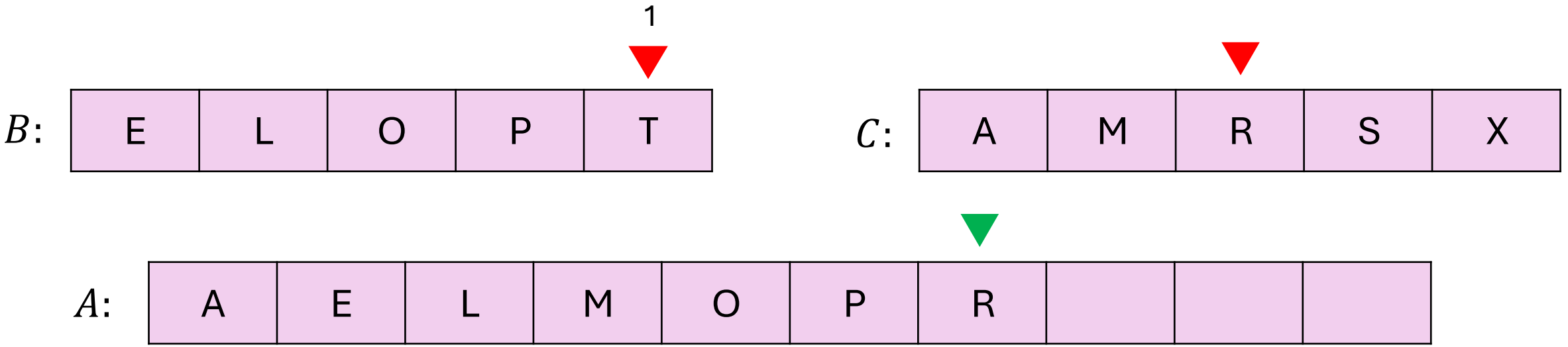
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 8+1

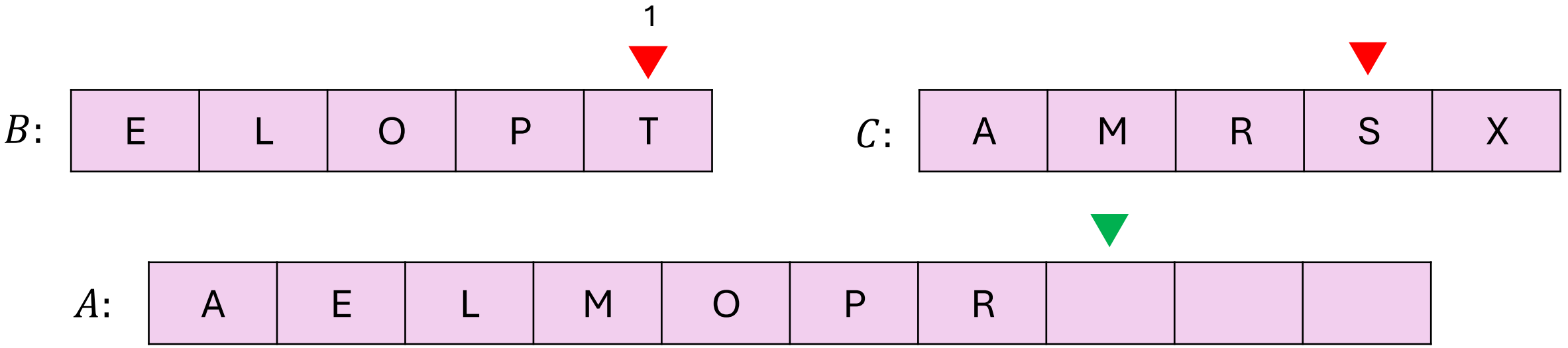
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 9

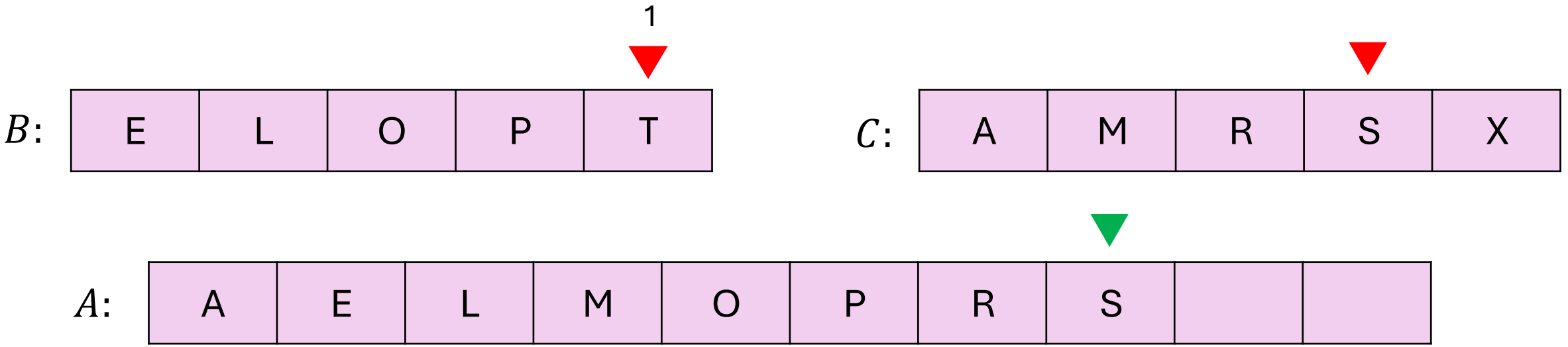
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 9

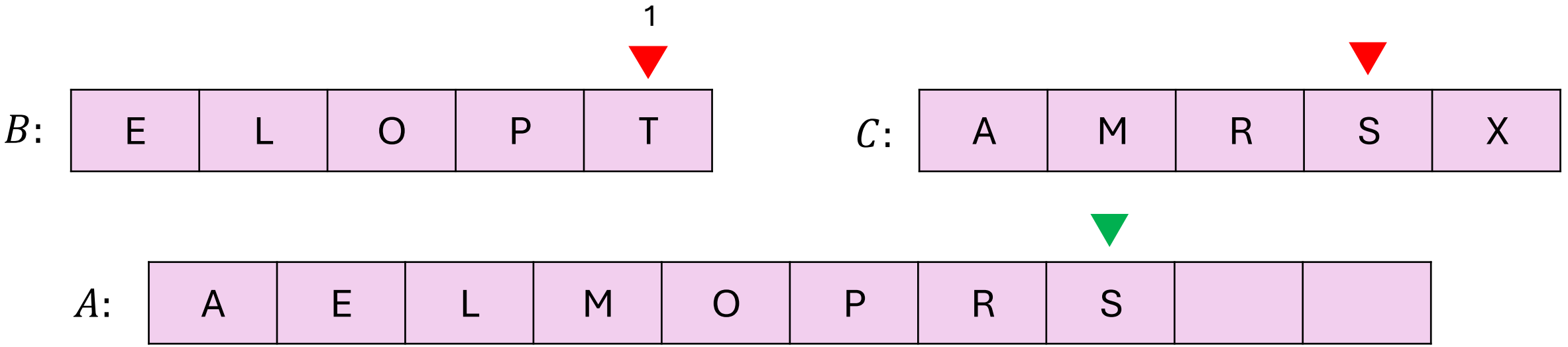
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 9+1

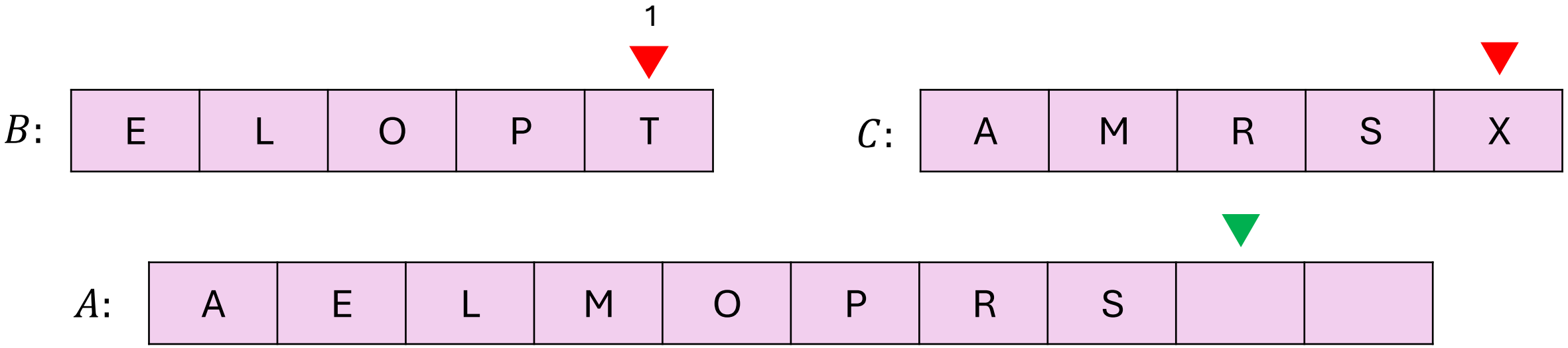
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 10

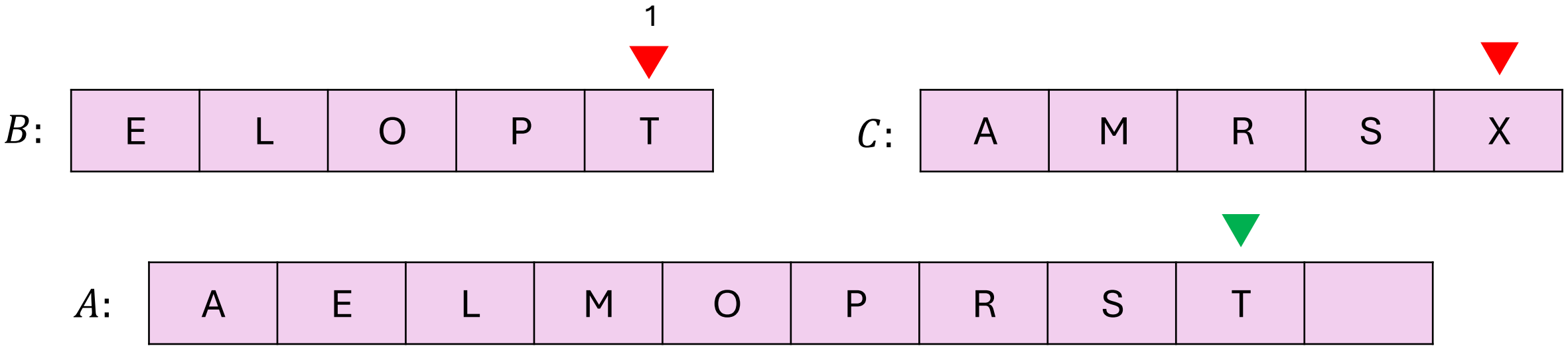
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 10

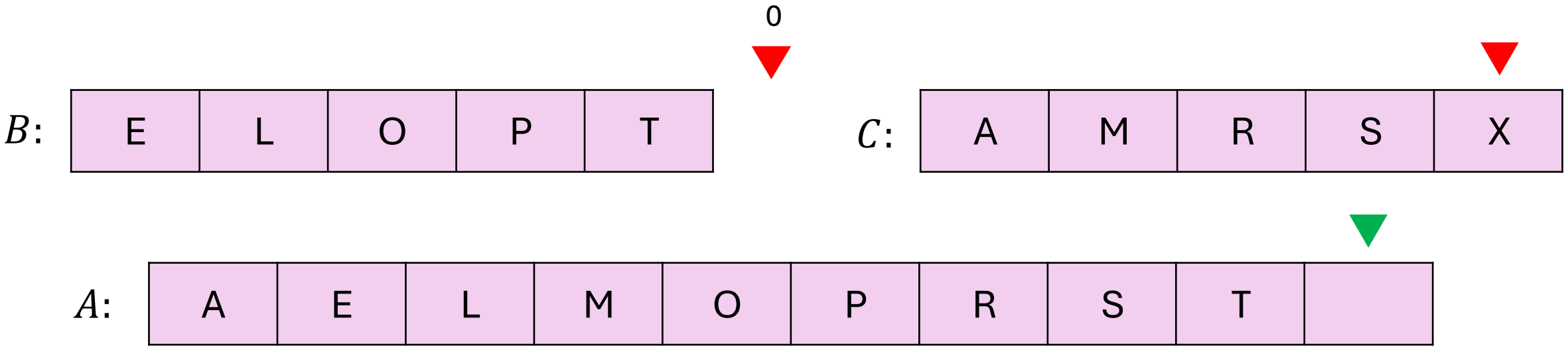
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 10

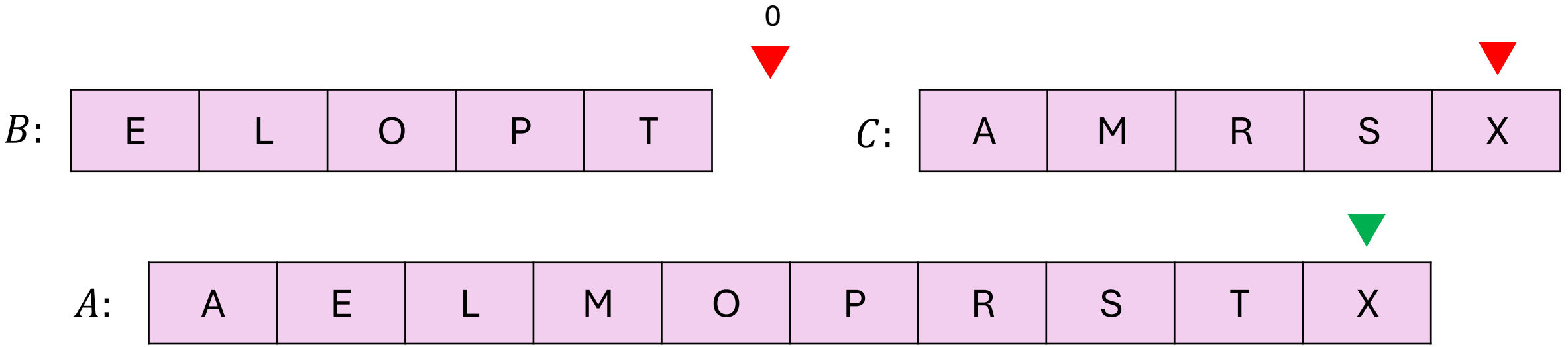
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 10+0

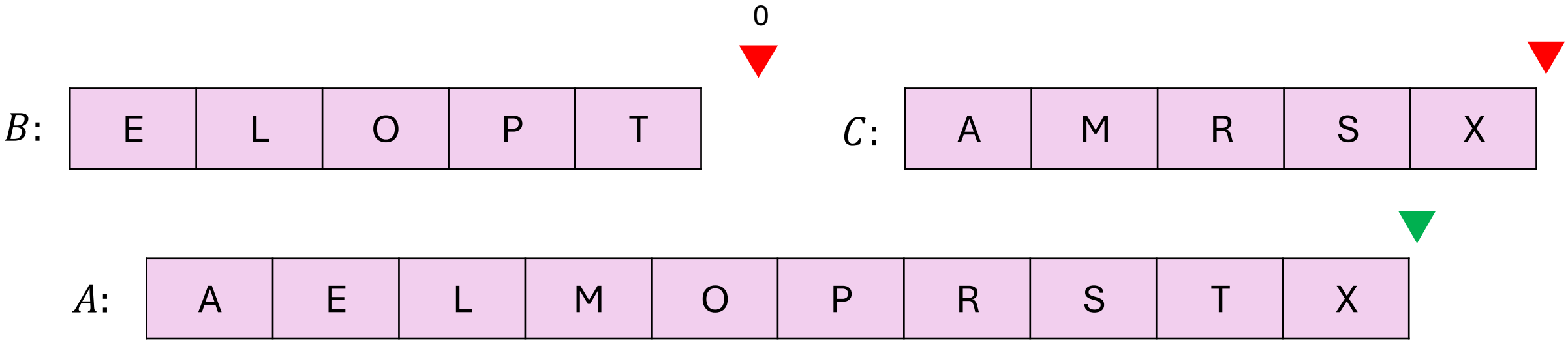
Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

First assume we have two (sorted) sub-arrays, namely B and C, that must be merged into a sorted array A.

How do we count the number of inversions between elements, where one is from B and one is from C?



#inversions: 10

Tutorial Questions

48. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Answer: We follow the hint that said to adapt mergesort. Let MERGESORT return the number of inversions that were in its input array (and as usual, have the side-effect of sorting it).

```
function MERGESORT(A[·], n) : Int
    if n > 1 then
        for i ← 0 to ⌊n/2⌋ − 1 do                    ▷ Copy left half of A to B
            B[i] ← A[i]
        for i ← 0 to ⌈n/2⌉ − 1 do                    ▷ Copy right half of A to C
            C[i] ← A[⌊n/2⌋ + i]
        b ← MERGESORT(B, ⌊n/2⌋)
        c ← MERGESORT(C, ⌈n/2⌉)
        a ← MERGE(B, ⌊n/2⌋, C, ⌈n/2⌉, A)
        return a + b + c
    else
        return 0
```

Tutorial Ouestions

```
function MERGE(B[·], p, C[·], q, A[·]) : Int
    i ← 0; j ← 0; k ← 0
    res ← 0
    while i < p and j < q do
        if B[i] ≤ C[j] then
            A[k] ← B[i]
            i ← i + 1
        else
            res ← res + p − i
            A[k] ← C[j]
            j ← j + 1
        k ← k + 1
    if i = p then
        for m ← 0 to q − j − 1 do
            A[k + m] ← C[j + m]
    else
        for m ← 0 to p − i − 1 do
            A[k + m] ← B[i + m]
    return res
```

Problem 6. Let $A[0..n-1]$ be an array of n integers. A pair $(A[i], A[j])$ is an *inversion* if $i < j$ but $A[i] > A[j]$, that is, $A[i]$ and $A[j]$ are out of order. Design an efficient algorithm to count the number of inversions in A.

Hint: Try to adapt mergesort for this problem, so as to achieve an algorithm with running time bounded by $O(n \log n)$.

Solution

Now that we know how to count #inversions in a 'merge' step, we can use recursion to count #inversions over an entire array A.

- Divide A into sub-arrays B and C (which may now be unsorted).
- Recursively sort B and C and, along the way, recursively count

  #inversions-in-B
  #inversions-in-C

- Merge B and C to sort A and, as shown previously, count #inversions-merge.
- To get the total #inversions-in-A, calculate

  #inversions-in-A = #inversions-in-B + #inversions-in-C + #inversions-merge